

A TVC256k++ kártya használata, programozása

2026.10.11.

"Nem bonyolult ám!"
(Magyar közmondás, 2026-ból)

—

Vass Sándor

Tartalomjegyzék

Tartalomjegyzék **1**
Áttekintés **6**
Kezdetek **6**
Fontos tudnivalók **7**
Firmware frissítés **7**
Fogalmak **8**
Bevezető **9**
Memória **11**
U2 és U3 területek 12
PSRAM 12
Grafika **12**
A képernyő 12
Scroll 14
Sprite-ok 14
Furcsa kivételek 15
A kirajzolás 15
File kezelés **16**
USB drive 16
SlowRAM avagy PSRAM drive 17
Flash drive 18
A flash drive otthoni feltöltése 18
DSK Drive 19
LOAD 19
SAVE 20
Könyvtárak 20
Programozás **20**
IO PORTOK 20
Portok szerepe 21
REGISZTEREK I. 21
RENDSZER FUNKCIÓK 22
REGISZTEREK II. 23
REG_SPRITE_X = $00 23
REG_SPRITE_Y = $20 (32) 23
REG_SPRITE_COLOR = $40 (64) 23
REG_SPRITE_PHASE = $50 (80) 23
REG_SPRITE_FOREGROUND = $60 (96) 23

REG_SPRITE_ENABLE = $70 (112) 24
REG_SPRITE_COLORMODE = $80 (128) 24
REG_SPRITE_ENABLE_LOW = $90 (144) 24
REG_SPRITE_ENABLE_HIGH = $91 24
REG_SPRITE_FOREGROUND_LOW = $92 24
REG_SPRITE_FOREGROUND_HIGH = $93 25
REG_SPRITE_COLORMODE_LOW = $94 25
REG_SPRITE_COLORMODE_HIGH = $95 25
REG_SPRITE_SP_COLLISION_LOW = $96 26
REG_SPRITE_SP_COLLISION_HIGH = $97 26
REG_SPRITE_BG_COLLISION_LOW = $98 26
REG_SPRITE_BG_COLLISION_HIGH = $99 26
REG_SPRITE_SP_IRQMASK_LOW = $9A 26
REG_SPRITE_SP_IRQMASK_HIGH = $9B 26
REG_SPRITE_BG_IRQMASK_LOW = $9C 27
REG_SPRITE_BG_IRQMASK_HIGH = $9D 27
REG_SPRITE_BASE_ADDR = $9E 27
REG_SCREEN_VIDEOMODE = $A0 (160) 27
REG_SCREEN_SCREEN_BASE_ADDR = $A1 27
REG_SCREEN_SCREEN_COLOR_BASE_ADDR = $A2 28
REG_SCREEN_FONT_BASE_ADDR = $A3 28
REG_SCREEN_BITMAP_BASE_ADDR = $A4 28
REG_SCREEN_SCROLL_X = $A5 29
REG_SCREEN_SCROLL_Y = $A6 29
REG_SCREEN_BORDER_COLOR = $A7 29
REG_SCREEN_Y = $A8 29
REG_SCREEN_MAXY = $A9 29
REG_MEMORY_P2 = $B0 (176) 29
REG_MEMORY_P3 = $B1 30
REG_MEMORY_MAP_8M_P2_LOW = $B2 30
REG_MEMORY_MAP_8M_P2_HIGH = $B3 31
REG_MEMORY_MAP_8M_P3_LOW = $B4 31
REG_MEMORY_MAP_8M_P3_HIGH = $B5 31
REG_MEMORY_PSRAM_SIZE_IN_MB = $B6 31
REG_MEMORY_ROM_PAGE = $B7 31
REG_FUNCTION_EXECUTE = $C0 (192) 31
REG_FUNCTION_PARAM_START = $C1 32
REG_FUNCTION_RESULT = $C2 32
REG_FW_VERSION_MAJOR = $C3 32

REG_FW_VERSION_MINOR = $C4 32
REG_DRIVE_SELECT = $C5 32
REG_FUNCTION_MULTI_EXECUTE = $C6 33
REG_FUNCTION_BITMAP_BASE = $C7 35
REG_DRIVE_PSRAM_VALID = $C8 35
REG_DRIVE_FLASH_VALID = $C9 35
REG_DRIVE_DSK_VALID = $CA 35
REG_USB_INIT = $D0 (208) 36
REG_USB_MOUSE_BUTTONS = $D1 36
REG_USB_MOUSE_DX = $D2 36
REG_USB_MOUSE_DY = $D3 36
REG_USB_MOUSE_DW = $D4 36
REG_USB_MOUSE_SPEED = $D5 36
REG_USB_MSC_CMD = $D6 36
REG_SID_BASE = $E0 (224) 37
TVC OS rendszer fgv hívások 37
GET_PWD - $D6 38
CH_DIR - $56 38
OPEN_DIR - $57/$D7 38
READ_DIR - $D8 39
CLOSE_DIR - $58 39
MK_DIR - $59/$D9 40
DELETE - $5A/$DA 40
RENAME - $5B/$DB 40
SEEK_FILE - $5C/$DC 41
GET_IOBASE - $5D/$DD 41
GET_MEMBASE - $5E/$DE 41
MOUNT DSK - $5F 41
UNMOUNT DSK - $DF 42
A REG_FUNCTION_EXECUTE függvény hívások 42
Param start 42
Execute és result 42
Általános rutinok 43
Clear_text_screen ($00) 43
Clear_bitmap_screen ($01) 43
Print_character_screen_code ($02) 43
Print_hex_byte ($03) 44
Print_hex_word ($04) 44
Print_string_screen_code ($05) 44

Print_string_ascii ($06) 44
Get_text_color ($07) 45
Set_text_color ($08) 45
Get_XY ($09) 45
Set_XY ($0A) 45
Memory_move_short ($0B) 46
Memory_move_full ($0C) 46
Memory_move_from_slow ($0D) 46
Memory_move_to_slow ($0E) 47
Copy_dir_to_psram ($0F) 47
Replace_pixel_color ($10) 48
Memory_move_chunks_from_block ($11) 48
Memory_move_chunks ($12) 49
Mirror_sprite_phase ($13) 49
zx7Decompress ($14) 50
Grafikai függvények 50
Get_pen_color ($15) 50
Set_pen_color ($16) 51
Get_dot_color ($17) 51
Set_dot_color ($18) 51
Draw_line ($19) 51
Draw_rectangle ($1A) 52
Fill_rectangle ($1B) 52
Draw_multi_line ($1C) 52
Scanline_flood_fill ($1D) 53
Draw_ellipse ($1E) 53
Fill_ellipse ($1F) 54
Copy_image_block ($20) 54
Copy_sub_image ($21) 54
PSRAM drive műveletek 55
Create_psram_drive ($22) 55
Get_first_usable_psram_pos ($23) 56
Delete_psram_drive ($24) 56
Egyszerű matematikai kifejezés kiértékelése 56
Evaluate_math_expression ($25) 56
Számok 56
Konstansok 57
Műveletek 57
Kiértékelés végrehajtása 58

Grafikai függvények II - két színnel 59
Fill_triangle_2c ($26) 60
Fill_box_2c ($27) 60
Scanline_flood_fill_2c ($28) 60
Memória blokkok másolásának kibővítése 61
Memory_move_chunks_2increments ($29) 61
File műveletek 61
Open_file ($81) 62
Close_file ($82) 63
Read_file ($83) 63
Write_file ($84) 64
Open_dir ($85) 64
Close_dir ($86) 64
Read_dir ($87) 65
Write_file_source ($88) 65
Read_file_dest ($89) 66
File_seek ($8a) 67
Getcwd ($8b) 67
Chdir ($8c) 67
Mkdir ($8d) 67
Delete ($8e) 68
Rename ($8f) 68
Getstat ($90) 68
Sync ($91) 69
Mount DSK ($92) 69
Unmount DSK ($93) 69
IO Műveletek emulátor készítőknek 70
Köszönetnyilvánítás **72**

Áttekintés

A kártya a Videoton TV Computer 8 bites személyi számítógép képességeit hivatott jelentősen kibővíteni az alábbi képességekkel:

- 16db sprite
- 256x240 pixel felbontású karakteres és grafikus képernyő
- 256kB + 8MB memória bővítő
    - 32kB -os RAM bővítőre nincs szükség, ne is használjunk mellette!
- USB pendrive és egér kezelés
- DSK image file kezelés
- 15MB flash terület rom drive-ként használva
- Korlátozott SID hangchip emuláció

Az alábbiakban a v0.24 firmware verziójú (vagy az utáni) kártya használatát ismertetem.

Kezdetek

Ha csak használni szeretnéd a kártyát, akkor nem kell mást tenned, mint bedugni a TVC felső, jobb szélső bővítő portjába, majd bekapcsolni a gépet (pendrive nem is kell, mert ilyenkor a beépített flash drive-ot használja a kártya).

Beírod, hogy `LOAD`, a betöltődött `LAUNCHER.CAS` program segítségével kiválasztod az elindítandó `.CAS` file -t, RETURN -t nyomsz rá és - amennyiben nem autostartos a kiválasztott programod - a `RUN` paranccsal futtatod.

Ha szeretsz gépelni, akkor a következőféleképpen kell eljárni:

A EXT parancsbővítővel kijelölni a megfelelő könyvtárat a flash drive-ról és onnan betölteni a kiválasztott játékot. Például, BASIC promptból el kell indítani az EXT CLI-t:

```
EXT 0
CD GAMES
X
LOAD "fuss!"
```

Az `EXT 0` parancsbővítő segítségével listázni is lehet a kiválasztott drive -ot, de akár drive -ot is váltani. A `HELP` parancs kiírja a lehetőségeket, ilyenkor érdemes `graphics 2` módban használni az EXT parancsbővítőt, mivel a képernyőn csak így fog elférni a `HELP` parancs szövege (a hagyományos CTRL-P működik).

Ennél sokkal többet tud a kártya, a részletekért tessék tovább olvasni!

Az `EXT` parancsot mindig azzal a sorszámmal kell paraméterezni, ami a felhasznált bővítő foglalat sorszáma: jobbról balra hanyadik foglaltban vagyunk, 0-val kezdve a számolást.

Fontos tudnivalók

a) A kártyát csak kikapcsolt TVC-be szabad bedugni, onnan kihúzni. A pendrive -t ki lehet húzni menet közben is, de óvatosan, mert az élcsatlakozó érzékeny.
b) A kártya nem kezeli azt a szituációt, amikor más IO kártya is szeretné használni az EXTC és az EC0-3 vonalakat a felső bővítőben. Értsd: a TVC256k++ kártya mellé ne rakj másik TVC256k++ vagy 2dfx kártyát, mert a TVC256k++ kártya tönkremegy.
c) A kártya egy memória bővítő is. Más RAM bővítő modullal nem használható egyszerre, mert tönkremehetnek
d) Az EXT *n* parancs hatására egy CLI nyílik meg előttünk. Az *n* paraméter mindig annak a foglalatnak a sorszáma, ahova a kártya be lett helyezve (jobbról balra, 0-3).
A HELP parancs szöveges leírást ad a kártya használatáról - angolul.
e) A kártyának szüksége van a felső bővítő csatlakozókra kivezetett /HSYNC, /VSYNC, BORDER és 6M25 jelekre. Ha a BORDER vagy a 6M25 jel nincs meg, akkor nagy valószínűséggel a TVC sem működik. Az én 64k+ gépemen a /HSYNC és a /VSYNC jel volt hibás (a C11 és a J22 -es IC-k cseréjével orvosoltam a problémát). Kompozit kábelen használom a gépem, semmi gond nem volt. Úgy gondolom, hogy ha egy TVC egy RGB monitoron (ahol kell a /VSYNC és a /HSYNC jel) nem ad képet, valószínűleg a TVC256k++ kártya sem lesz üzemképes.

Firmware frissítés

Ha esetleg egy újabb (vagy régebbi) firmware -t szeretnél felrakni a kártyára, akkor

- Vedd ki a TVC-ből a kártyát
- Csatlakoztasd a PC-dhez a kártyán található USB-C (pici, kerekített) foglalaton keresztül. A PC-den valamilyen USB foglalatba kell dugni a kábelt.
- Nyomd meg a kártyán a BOOT és a RST gombokat egyszerre (ha a kábel felül van, akkor a gombok alul: bal szélső és középső gomb)
- Engedd fel a RST (középső) gombot - de csak azt!
- Majd engedd fel a BOOT (szélső) gombot is.

Ekkor megjelenik egy új, hordozható drive a PC-den. Ide kell bemásolni a kedvenc file kezelőddel a .uf2 kiterjesztésű firmware állományt. A drive kisvártatva eltűnik a PC-ről, a firmware frissítés elkészült.

Az új firmware verzióját TVC alól, `EXT` CLI-ben a `ver` paranccsal ellenőrizheted.

Fogalmak

- RAM és ROM
  Mindkettő kifejezés memóriát jelent, a különbség közöttük, hogy a RAM az változtatható, a ROM pedig *nem* változtatható memória (sőt, kikapcsolás után is megőrzi tartalmát).
- P2 és P3 memória területek a TVC-ben
  A TVC számítógép Z80 processzora 64kB memóriát képes címezni egyszerre. Az, hogy a címzett memória RAM vagy ROM, az a processzor környezetétől - itt a TVC számítógéptől - függ. A TVC mérnökei a 64kB memória területet 4db 16kB -os tartományra, lapokra osztották. A 32k gépekben csak az első két lap érhető el. Ezeket a lapokat (page-eket) nevezzük sorrendben P0, P1, P2 és P3 -nak.

  Kis érdekesség: a TVC felső bővítőjére ki van vezetve a /P2 és /P3 jel (alacsony-aktív). A TVC mérnökei itt egy apró szemantikai hibát követtek el, ezeknek a jeleknek a nevei ugyanis /U2 és /U3 -nak kellett volna lenniük. Ezek a jelek ugyanis azt jelzik a csatlakozóba dugott IO kártya felé, hogy az adott lapon a TVC számítógép RAM területe van címezve. A TVC szakirodalomban ugyanis a 64kB RAM terület is 4 részre van felosztva és azokat U0, U1, U2 és U3-nak nevezzük.
  Például a /P2 nem aktív, ha a TVC a $8000 -as területre a videoRAM -ot lapozta be és a CPU a $8000 -es címen lévő byte -ot címez
- Sprite
  a képernyőn megjelenített grafikai objektum, aminek pozíciója, kinézete független a képernyőn megjelenített háttértől, karakterektől (többi sprite-tól).
  Gyári állapotában a Videoton TVC számítógép nem tud sprite-okat megjeleníteni.
- MCU
  Micro Controller Unit, mikrokontroller. A kártya központja egy RP2350B mikrokontroller (a RaspberryPI cég készíti, de ez nem az a ‘CPU’, ami a Raspberry PI SBC -ken található). Ez az egység gyakorlatilag egy miniszámítógép 2 ARM processzor maggal, RAMmal, perifériákkal (DMA, PIO, timer, GPIO) és flash memóriával. Ezen a 'gépen' is egy program fut ezt hívjuk firmware -nek.
- Belső RAM, gyors RAM, lassú RAM/PSRAM
  A TVC256k++ kártya memória bővítőként is viselkedik, tudja szabályozni hogy a P2 és a P3-as területeken a Z80 processzor milyen RAM-ot lásson.
  A TVC saját RAM-jára belső, vagy rendszer RAM-ként hivatkozok a dokumentum további részében.
  Gyors RAM-nak nevezzük a TVC256k++ kártyán található MCU belső RAMját, az innen belapozott RAM területet gyorsan lehet cserélni, változtatni, belapozás esetén a Z80 CPU közvetlenül el tudja érni.

Lassú RAM, PSRAM (Pseudo Static RAM). Ez egy olyan RAM terület, ami kívül helyezkedik el az MCU -n, elérése sokkal lassabb, több időt vesz igénybe, cserébe jóval nagyobb területet kínál.

Bevezető

A kártya agya egy RP2350B jelű mikrokontroller. Ez az MCU a következő dolgokért felel:

1. a Z80 CPU buszkezeléséért
   a. Amikor a Z80 memória hozzáférést kezdeményez, akkor a TVC - az aktuális beállításoktól függően - tudja jelezni az IO kártyának, hogy RAM hozzáférés történik-e éppen a P2 vagy a P3 lapokon. Ezt a kártya érzékeli és - szintén beállítástól függően - a saját, belső RAM-jából ajánl fel memória területet a TVC számára.
   b. Amikor a TVC a kártya ROM-ját olvassa (rendszer induláskor vagy rendszer fgv-ek kezelésekor), a kártya az erre dedikált memória területét nyújtja a TVC felé, a TVC azt fogja tudni olvasni, onnan utasításokat végrehajtani. Olyan ez, mintha egy ROM chip is lenne a kártyán.
   Érdekesség: a kártya IO ROM területének utolsó 2kB -ja valójában RAM, amit a kártya a saját rendszerváltozóinak használ. Ezt a területet felhasználói programoknak nem illik használnia.
   c. Amikor a Z80 CPU IN/OUT utasítást hajt végre a kártya slot -jának megfelelő port tartományban, a kártya ezeket a műveleteket is 'elfogja' és értelmezi, végső soron a kártya belső regisztereinek állapotait lehet így változtatni.
   Érdekesség: a kártya érzékeli a $00 portra írást (border szín) és a saját, 8 pixel széles (scroll miatti bővített) keretének színét erre a színre állítja.
   d. Amikor a T. felhasználó RESET gombot nyom a TVC alján, akkor ezt a jelet is elfogja az MCU.
   A TVC gépen kétféle reset van: warm (meleg) és cold (hideg) reset. Ha a gép alján található reset gombot egyszer nyomjuk meg, akkor warm reset indul (ilyenkor nem indul teljesen újra a gép, csak a kulcsfontosságú rendszerváltozók inicializálódnak újra és kapunk egy üres képernyőt az OK prompttal), bekapcsoláskor vagy ha gyors egymásutánban kétszer nyomjuk meg a reset gombot, akkor indul a cold reset.
   Mindkét esetben a Z80 reset lábát - és így a csatlakozó portban lévő vezetéket is - lehúzza a gomb megnyomása. A kártya figyeli ezt a vonalat, de neki saját maga kell eldöntse, hogy most hideg vagy meleg reset hajtódik-e végre.
   A helyzetet tovább bonyolítja az is, hogy bizonyos esetekben a meleg resetet nem lehet végrehajtani, ezért a TVC OS automatikusan hideg resetet hajt végre - egy gombnyomásra. Ezt a szituációt a kártya egyáltalán nem érzékeli.
   A dupla reset-lenyomást viszont igen, de csak egy saját időzítőt néz a kártya, továbbra sem tudományos módszerrel. Amennyiben a reset gombot 1

másodpercen belül kétszer megnyomja a kedves felhasználó, akkor az MCU egy teljes reset ciklust hajt végre.

Ha egy másodpercen belül csak egyszer nyomunk gombot, akkor a kártya meleg resetet hajt végre és csak

- az aktív videoMode -t visszaállítja 0 -ra (nincsenek a képernyőn grafikai elemek)
- elrejti a sprite -okat
- a képernyő magasságát visszaállítja 240-re
- a képernyő eltolást alaphelyzetbe állítja, bővített kereteket kikapcsolja
- elcsendesíti a hangot (a #24-es vSID regiszterbe 0-t ír)

Figyelem! File-okat nem zárja be, lapozást nem változtat, gyors és lassú RAM tartalmát nem törli. De vigyázat! A TVC minden resetkor használja az U2-t, azaz a memória tartalma ott változhat a TVC ROM miatt!

"Nem tudományos", értsd: néha sajnos hibázik. Ilyenkor meg kell ismételni a dupla reset gombnyomást!

2. a TVC grafikai overlay kirajzolásáért
   A TVC256k++ kártya minden esetben (TVC felbontásától függetlenül) egy 256x240 felbontású képet rajzol a TVC képernyője fölé.
   Ezen az overlay képen sprite -ok, karakterek és bitmap grafikát jelenít meg. Az MCU a kivezetett 6.25MHz -es órajel, a BORDER és a /HSYNC jelek segítségével tud szinkronizálni a megjelenített képernyővel.
   a) Amennyiben a képernyő magasságát csökkentettük a CRTC regiszterbe, akkor a kártya 240 -es magasságát is csökkentenünk kell REG_SCREEN_MAXY regiszter segítségével
   b) Ha a képernyő szélességét csökkentjük a CRTC regiszterek segítségével, akkor a kártya nem fog helyesen működni. A kártya elvárja, hogy legalább 256 megjeleníthető pont legyen a képernyőn (az ő felbontásában).
3. USB device -ok kezeléséért
   A kártya limitált USB Host vezérlőként is működik. Egyetlen USB eszközt tud kezelni (USB Hub -ot nem támogat), ami lehet pendrive (FAT16/32, exFat filerendszerrel, hosszú fileneveket) vagy USB egér. A kártya low- és highspeed USB eszközöket tud kezelni (USB 1.1).
   USB-s gamepad-okat natívan nem támogat. Amikkel kipróbáltam, azok azt vallották magukról, hogy joystick-ok és 3 gamepad 3 féle képpen jelentette a le a gombjai, DPAD állapotát. Ebbe az irányba én nem indultam el.
   Kerüljük a nagy áramfelvétellel járó eszközök használatát, pl. ne erről az USB portról töltsük telefonunkat: ez megterhelheti a TVC tápot és alaplapot egyaránt.
4. Flash filerendszer
   A modul 16MB flash területéből ~1MB van lefoglalva a kártya adatai számára. A maradék 15 MB-on egy ún. Flash file rendszer található. A TVC innen is tud file-okat olvasni, programokat betölteni. Írásra, file-ok változtatására TVC-ből nincs lehetőség (külső PC -ről viszint igen).
5. hang generálás

A kártya egy SID hangchipet is emulál, limitált módon. 3 hangcsatorna, csatornánként ADSR envelope, 4 féle hullámalak, ring moduláció, sync és a test bitek is implementálva vannak. A SID emulációt vSID -nek nevezem (virtual SID). A vSID 985337Hz -en fut (szemben a PAL rendszerű C64-el, ami 985248Hz -en dolgozik). A két freq között kb 0.01% eltérés van, azaz nyugodtan lehet használni azokat a frekvencia értékeket a vSID emuláción, mint a C64-en.

A hangot egy 1.3125MHz-es PWM jel állítja elő, 16 bit felbontásban (2db 8bites PWM kimenet). Az egyes SID hangmintákat az MCU 41kHz-el számítja ki. Mivel nem analóg jelről van szó a kvantálásból adódóan könnyen előfordulhatnak olyan hangbéli eltérések, amik a valódi SID (vagy helyes emuláció során) esetén nincs.

Figyelem: a vSID nem törekszik a SID tökéletes emulációjára! Mindenki fogadja el úgy, ahogyan van (egyébként a robocop3 ezen kártyán egész pöpecül szól, tessék összevetni pl. a picoSID -del...). Ezért pl nincs megvalósítva a SID filterezés sem.

Memória

A TVC256k++ egy memória bővítő modul is a TVC számára. 4 fő memória területet tud megosztani a TVC-vel:

1. 256kB RAM. Ez a 'gyors' RAM. Belapozása 16kB-onként történik és ebből kiszámítható, hogy összesen 16db lap áll rendelkezésre. Belapozása azonnal megtörténik a parancs kiadásakor. Ezt a memóriát az MCU belső RAM-ja szolgáltatja.
2. 12kB ROM. Ez a TVC256k++ kártya ROM-ja. Ez felhasználói programoknak nem igazán érdekes terület. Az MCU belső RAMjában egyszerre csak 6kB van bemásolva (belapozva) a 12kB-ból, illetve további 2kB pedig RAM-ként funkcionál a ROM rutinoknak. 6+2kB, így jön ki a 8kB -os IO ROM terület. A 2kB RAM nem lapozódik.
3. 8MB RAM. Ez a 'lassú' RAM. Belapozása szintén 16kB -onként lehetséges, viszont az csak késleltetve történik meg a kártya leterheltségétől függően. Kiszámolható, hogy 512 lap áll rendelkezésre, a lapozási regiszter két byte-on van tárolva.
   Ezt a memóriát az ún. PSRAM IC adja. Felejtő és nem párhuzamosan elérhető RAM, elérése sokkal lassabb, mint az a Z80 CPU-nak elegendő lenne, így a kijelölt 16kB -ot használat előtt be kell másolni az MCU belső memóriájába.
   Van egy felhasználási módja: lehetőség van egy könyvtár tartalmát (file-okat) bemásolni ide egy pendrive-ról és onnantól a TVC file műveletei ezeket a file-okat látják. Az USB port pedig felszabadul, mondjuk egér kezeléshez.
4. 15MB flash ROM. Ez a mikrokontroller firmware melletti ‘maradék’ ROM. Itt egy - a TVC számára - csak olvasható filerendszer lehet. Ez a flash drive.

E felsorolt memóriaterületeket természetesen a TVC csak akkor látja, ha a $02 -es port is megfelelően be van állítva: a RAM elérésekhez U2 és/vagy U3 a megfelelő lapon, illetve az IO ROM eléréshez EXT (a 3. lapon és a megfelelő kártya kell legyen becsatolva a $03-as porton).

U2 és U3 területek

Az U2 ($8000-$BFFF) és az U3 ($C000-$FFFF) RAM területek helyére külső gyors és lassú RAM lapozható be. Bekapcsolás után az U2 és az U3 területeken a 4. és az 5. gyors RAM lap érhető el, a TVC már azt inicializálja, oda tölti a stack -et, stb..
Ebből az is következik, hogy 32kB -os gépek bekapcsolás után rögtön 64k gépeknek látszódnak.

Lehetőség van a memóriabővítés kikapcsolására is, ilyenkor 255 -öt kell írni a megfelelő regiszterbe s így a TVC saját RAM-ja látszódik (ha van). BASIC esetén a lapváltás nem triviális, mivel a BASIC stack az elérhető memória végén van s ha azt kilapozzuk, a BASIC programunk megromolhat... Amit lehet tenni, hogy a BASIC területet visszavesszük a $7FFF végére (POKE 2842, 128) és kiadunk egy NEW parancsot. Ilyenkor a BASIC nem használja a P2 lapot és innentől azt kedvünkre lapozgathatjuk ki-be.

Látni fogjuk majd, hogy az MCU első 4 lapján található gyors memória szerepe speciális: a grafikai objektumok megjelenítéséhez a kártya innen veszi az adatokat: bitmap, font, sprite-ok képei, képernyő karakterei, színei.

Bekapcsoláskor a demonstrációk alkalmával számtalanszor bemutatott mikulássapi, a havas táj képe és a magyar ékezetes karakterekkel kibővített C16/plus4 font készlet bemásolódik ide.

PSRAM

A PSRAM (vagy ahogy néha hivatkozok rá: lassú vagy slow RAM) kijelölésével elindul a belapozási folyamat.

Ha előtte egy másik lassú RAM lap volt belapozva, akkor azt először vissza kell másolni a PSRAM-ba, majd utána a kijelöltet bemásolni az MCU belső RAM-jába. Erre a célra egy-egy külön lap van szánva az MCU belső RAMjából, ezek a 16. és a 17. lapok (U2 és U3 esetben).

Ezeket nem lehet kézzel kijelölni, viszont ha PSRAM van bemásolva, akkor az aktuális U2 vagy U3 lap regiszter olvasásakor 16 illetve 17 értékeket kapunk attól függően, h mely területre lapoztuk be a PSRAM-ot. Ha U2 -re PSRAM-ot lapoztunk be, majd oda a gyors RAMból akarunk memóriát belapozni, nem történik másolás, a belapozás azonnal megtörténik. Ha ugyanide, ugyanazt a PSRAM lapot akarjuk visszalapozni, akkor sem történik másolás, viszont az időbe telik, mire a megfelelő regiszter-értékek beállnak, ilyenkor SEM azonnali a lapozás! Ezt a RAM területet ezért is hívom néha slow-nak - lassúnak.

Grafika

A TVC256k++ kártya sprite-ok (fix, a háttérgrafikától független grafikai blokkok) megjelenítése miatt készült el. Ugyanakkor képes megjeleníteni egyéb grafikai elemeket, a TVC által generált kép fölött, azaz a TVC256k++ grafikája minden esetben a TVC képe fölött lesz látható.

A képernyő

A generált kép 256x240 méretű. A magasság módosítható érték, ha kisebbre vesszük, akkor a kártya csak a beállításnak megfelelő számú sort számol és rajzol ki. Ez utóbbinak

akkor van jelentősége, amikor a CRTC -ben egy szűkebb képernyő magasság van beállítva (pl a játéktér keskenyebb). Ezt a CRTC beállítást követnie kell a kártyának is.

Az overlay képernyő szélessége nem változtatható, az fix 256 pixel. Amennyiben ettől eltér a CRTC beállítás, az kártya által rajzol képernyő szét fog esni.

A grafikus elemeket a kártya a gyors RAM első **128kB** -jából ($00000 - $1FFFF) veszi (többnyire, de van kivétel, lásd később).

A kártya 4 grafikus módot támogat:

1. nincs grafika

   A kártya ilyenkor csak a sprite -okat jeleníti meg a képernyőn - amennyiben azok engedélyezve vannak
2. 2 színű karakteres képernyő

   A kártya ilyenkor kétszínű fontkészletből veszi a karakterek képeit, a screen RAM területről a kirajzolandó karakterek kódjait és a screenColorRAM területről pedig az egyes karakterek színeit.

   Egy karakter 8x8 pixelből áll és a képét 8 byte határozza meg. Az MSBit a karakterkép bal oldalán, az LSBit pedig annak jobb oldalán lesz megjelenítve (ahogyan a C= gépeken). Bekapcsolás után a font definíció a $001000 címen kezdődik.

   A megjelenített karakter kiválasztása a screen RAM területről történik. Ez egy 32x30 byte nagyságú terület és bekapcsolás után a $00400-as címen kezdődik.

   A megjelenített karakter színének kijelölése a screencolor RAM területről történik. Mérete és felosztása megegyezik a screen RAM méretével. Minden egyes byte alsó 4 bitje kijelöl egy színt IRGB sorrendben. (0 fekete, 1 s.kék, 2, s.zöld, stb...).

   256 különböző karaktert lehet megjeleníteni, a font definíció ezért 256x8=2kB, a screen RAM és a screencolor RAM is 32x30=960 byte területet igényel.
3. 16 színű karakteres képernyő

   Ugyanúgy karakteres képernyőt jelenítünk meg (screen RAM), de a font definíció komplexebb karakterképeket enged megjeleníteni.

   Egy karakter ugyanúgy 8x8 pixelből áll, viszont mind a 64 pixel színét a fontdefiníció tartalmazza. A karakter definícióban egy sornyi (8 pixel) képét 4 egymás melletti byte határozzal meg: `IRGBIRGB IRGBIRGB IRGBIRGB IRGBIRGB` definícióval. Az átlátszó pixelt az `0b1000` IRGB szín adja (azaz a 8-as szín).

   Mivel egy karakter egy sorát 4 byte határoz meg, a karakter 8 sorból áll és 256 karakterdefiníciónk van a fontkészletben, ebből az adódik, hogy a 16 színű font mérete 4x8x256=8kB.
4. 16 színű bitmap képernyő

   Ez az üzemmód egy 256x240 pixeles felbontású képet jelenít meg a kijelölt memóriaterületről. Egy pixelt a fent látott IRBG bitnégyes határoz meg, egy sort ezért 128 byte -on lehet tárolni. Ebből következik, hogy egy ilyen bitmap a memóriában 32kB helyet foglal.

   A bekapcsolás utáni bitmap terület a $08000 címre van állítva, ide másolódik be a havas táj is. Ugyan a havas tájon nincs ilyen pixel, de az átlátszó pixel a 0b1000 (8) -as szín.

Scroll

Lehetőség van a képernyő grafikájának az eltolására 0-7 pixellel vízszintes és/vagy függőleges irányban. Mivel a scrollozás, mint effektus új karakterek behozatalát jelenti ezért lehetőség van arra is, hogy 8-8 pixel széles, bővített kerettel eltakarjuk a képernyő alsó-felső, illetve oldalsó széleit.

A bővített keret színe a $00 portra írt keret színe, de ez megváltoztatható egy TVC256k++ regiszter írásával is.

Sprite-ok

A kártya egy sorban 16db sprite megjelenítését teszi lehetővé. Egy sprite minden esetben 24x21 pixel méretű. Nincs pixel duplázásra lehetőség.

Egy sprite lehet 2 színű (ekkor a színt az adott sprite szín-regisztere határozza meg) vagy lehet 16 színű. Ilyenkor a pixelek színeit a már fent látott módon a sprite definíció egyértelműen meghatározza. Az átlátszó pixelt itt is a `0b1000` (8) 'szín' jelenti.

A sprite -ok pozíciója a (0;0) ponton kezdődik. Ez a bal felső sarokban van, a keret alatt. Ha egy sprite-ot ide pozicionálunk, akkor abból semmi sem látszódik. A (24;21) -es pont jelenti a látható (nem keret) képernyő bal felső sarkát, az ide pozicionált sprite teljes képe látszódni fog - amennyiben nincs bekapcsolva scroll miatti bővített keret.

A képernyő jobb alsó sarka - alap beállításoknál - a (279;260). Ezen a pozíción a sprite -ból még éppen látszódik a bal felső pixel.

A (280;261) pozícióban pedig a sprite képernyő kerete alatt van teljes terjedelmében. Látható, hogy a sprite-ok X és Y koordinátái nem férnek el 8 biten, ezért azok 16 biten tárolódnak. Nagy értéket be lehet állítani, de felesleges.

Egy sprite vagy a grafikai elemek előtt, vagy mögöttük rajzolódik ki. Ha a grafika mögött van, akkor a sprite csak a grafika transzparens pixelei 'alatt' látszódhat. Ha a grafika előtt rajzolódik ki, akkor a sprite nem transzparens pixelei kitakarják a mögötte lévő grafikát.

Egymás között a nagyobb sorszámú sprite látszódik előrébb, az takarja ki a kisebb sorszámú sprite pixeleit, de csak akkor, ha a grafikához képest ugyanaz az elhelyezkedésük. Egy sprite, ami a grafika előtt jelenítődik meg, az minden olyan sprite -ot takarni fog, amik a grafika mögött jelennek meg. Ez azért van, mert az MCU ebben a sorrendben számolja ki a megjelenítendő pixeleket egy adott sorban:

1. Először feltölti a kirajzolandó sort üres (átlátszó) pixelekkel
2. Erre felrajzolja azon sprite -ok pixeleit, melyek ebben a sorban láthatóak és a foreground flag 0-ra van állítva
3. Ez után felmásolja az adott sor képernyőn látható grafikai elemek pontjait - ha van ilyen
4. Majd legvégül azon sprite-ok pixeleit, melyek ebben a sorban láthatóak és a foreground flag 1-re van állítva

A megjelenített sprite alakját a sprite phase regisztere határozza meg. 2 színű üzemmódban a 24x21 pixelt egy 3x21 byte méretű terület, 16 színű üzemmódban pedig egy 12x21 byte méretű terület határoz meg. A phase regiszter azt jelöli ki, hogy melyik beállított sprite területtől kezdve melyik fázist, alakot mutassa a kártya. Bekapcsoláskor a sprite terület kezdete a `$00000` -ra van állítva.

Például egy kétszínű sprite esetén a 2-es fázis beállítása a memória `$00080` területtől kezdődő 63 byte határozza meg a kirajzolandó sprite képét. Egy 16 színű sprite 2-es fázisa a memória `$00200` területétől.

Mind a 16 sprite külön-külön, ki- és bekapcsolható (enable).

Mind a 16 sprite-ra külön-külön beállítható, h a grafika előtt vagy mögött látszódjon (foreground)

Mind a 16 sprite-ra külön-külön beállítható, h 2 vagy 16 színű legyen (colormode).

Mind a 16 sprite-ra külön-külön beállítható, h milyen alakot mutasson (phase).

Mind a 16 sprite-ra külön-külön állítható az X és az Y koordináta (16 bites regiszterek).

Furcsa kivételek

A fontkészlet és a sprite alakok kezdőcíme a memórián belül úgy választható ki, hogy a 2 színű üzemmódban egész számúan elférjen a 128kB-ban, az első 8 lapon, átfedések nélkül.

A fontkészlet kezdőcíme 2kB-onként választható ki (a 16 színű fontok esetén is), így 64 különböző értéket lehet beállítani. Amennyiben viszont a font kezdőcíme `$3f`-re van állítva (ez jelenti a `$1f800` -as font kezdőcímet a memóriában) és 16 színű font video üzemmódra kapcsolunk, akkor a fontkészlet az `$1f800-$21800` területről lesz felhasználva.

A különböző 16 színű fontkészletek egymással átlapolódhatnak.

Ugyanígy, a sprite-ok címe helyről kezdődhet a memóriában: $00000, $04000, stb.... Ugye 256 különböző fázist tud megjeleníteni a kártya, 64 byte-tal számolva az pont egy lapnyi, $4000 nagyságú terület.

Ha úgy állítjuk be a kártyát, hogy a sprite képernyőadatait a 7. lapról olvassa be és 16 színű sprite-okat mutatunk, akkor a fázis regisztert 256-al kell szorozni, így a sprite-ok a $01C000-$02C000 (gyorsRAM) tartományból kerülnek kirajzolásra.

A kirajzolás

Megj: Az ebben a fejezetben leírt időzítéseket képernyő kirajzoló raszter sugárnak megfelelően írom le. Amikor azt írom, hogy a képernyő kezdete előtt kell megtörténjen valami, akkor azt úgy értem, hogy a számolásnak akkor kell megtörténnie, amikor a rasztersugár még a keretet rajzolja, a képernyő kirajzolása előtt.

Az MCU az ún. PIO modult használja, hogy a képernyőre rajzoljon. Technikai részletektől megkímélem a kedves olvasót, annyit azonban érdemes tudni, hogy az MCU egyik ARM magja azzal foglalatoskodik, hogy az overlay képernyő sorokat (amik a natív TVC képernyő sorai fölött látszódnak) előállítsa, majd egy belső DMA csatorna segítségével a PIO a

6,25MHz -es órajel, a BORDER és a HSYNC jelekkel szinkronban beállítja az overlay grafika megjelenítésére alkalmas 5 vezérlőjelet a felső csatlakozóban (/EXTC, EC0, EC1, EC2 és EC3). A képernyő sorok előállítása, annak időzítése valahogy így zajlik:

1. minden pixel 1 byte (`---eirgb` alakban, ahol 'e' az /ENABLE bit, azaz kell-e rajzolni pixelt vagy sem, irgb pedig a szín) formában kell előállítani. Mire a rasztersugár elkezdi kirajzolni a képernyő pixeleit, a pixeladatoknak már készen kell lennie. Általánosságban elmondható, hogy az `N.` képernyő sort az `N-1.` sor kezdetén (amikor a bal oldali keret végetér és kezdődik a képernyő) kezdi el kiszámolni az ARM mag.

   Az `N.` sorban megjelenítendő objektumok attribútumai az `N-1.` sor bal oldali keret vége előtt be kell állítva legyenek.

   Ha a sprite-ok adatait vagy a karaktereket csak az `N-1.` sorban állítjuk be (a bal oldali keret után), akkor azok hatása az N. sorban **nem** fognak látszódni (vagy előfordulhat, h csak töredékesen).

   A PIO programjának teljes képernyősorra van szüksége, ezért képernyő sorokat számol az ARM mag és nem különálló pixeleket ad a PIO modulnak.
2. A kártya a 0-20 képernyősorokat nem számolja, a képernyőn az első látható sort, a 21. képernyősort pedig közvetlenül a HSYNC jel vége után számolja ki, azaz a fölső keret kezdetekor. Ha valamit ott, mármint az első látható képernyősorban szeretnénk megjeleníteni, akkor azt még a HSYNC jel előtt be kell állítani (mondjuk az alsó keret kezdetekor).

   A 22. sortól igaz a fenti `N-1` -es gondolatmenet, azaz a 22. sorban (a képernyő 2. látható rasztersorában) a megjelenítendő elemek attribútumai már be kell állítva legyenek akkor, amikor az első látható képernyősor elkezdődik kirajzolódni.

   Ugyanezért, ha beállítjuk a keretszínt a $00 portra írással, akkor a bővített keret színe nem abban a sorban fog megváltozni ahol a $00 portra írtunk, hiszen az adott sor képernyő-adatai már készen vannak és a HW éppen jeleníti meg azokat egy belső bufferből.

File kezelés

A TVC256k++ kártya egyik nagyszerű tulajdonsága, hogy képessé teszi a TVC számítógépet programok, adatok egyszerű betöltésére, kimentésére. A kártya négy meghajtót, drive -ot tud használni: USB drive, slowRAM drive, flash drive és felmount .dsk image -t DSK Drive-ként lát.

Mivel a TVC OS csak legfeljebb 15 karakter hosszúságú filenevet kezel, ha mi magunk hozunk létre file-okat USB -n, flash drive -on, érdemes erre odafigyelni. Azonban van mód a hosszú nevű file-ok betöltésére is, de akkor az első 15 karakter nem egyezhet más file nevével: használni kell a * karaktert betöltéskor. Ezt a trükköt használja a LAUNCHER is, ha a betöltendő file neve túl hosszú.

USB drive

USB pendrive -ot csatlakoztatva a kártyához, az képes arról file-okat beolvasni, vagy azon file-okat létrehozni vagy programokat betölteni esetleg kimenteni. Az usb drive írható drive, azaz rá tudunk file-okat menteni, rajta könyvtárakat létrehozni.

Ha egy pendrive be van dugva a kártyába és úgy kapcsoljuk be a TVC számítógépet, akkor a kijelölt drive az USB drive lesz. Minden file művelet az USB drive-ra fog irányulni: onnan lehet file -t betölteni vagy arra lehet file -t kimenteni.

Megjegyzés: Az, hogy éppen melyik drive van kiválasztva az `EXT` parancsértelmező `DS` parancs tudja megmondani és ugyanezen parancs helyes paraméterezésével lehet bármikor drive -ot változtatni/választani. Pl.: `DS U`

Újból felhívnám a T. felhasználó figyelmét arra, hogy kerülje a nagy áramfelvétellel járó USB eszközök használatát.

SlowRAM avagy PSRAM drive

Lehetőség van arra, hogy a pendrive-ról egy teljes könyvtár tartalmát betöltsük és elraktározzuk a kártya PSRAM-jában. A 8MB -os PSRAM kikapcsolásig ezt az adatot meg is őrzi. A PSRAM drive szintén írható/olvasható drive, mint az USB drive.

Tartalmát kilistázhatjuk az EXT CLI segítségével, onnan .cas file-t be tudunk tölteni.

Ez a funkció lehetőséget biztosít arra, hogy az utántöltős játékok is tudjanak egeret használni (kiváló példa erre a Lemmings).

A gyorsítótárazást három féle képpen tehetjük meg:

- amennyiben a kártya és a TVC pendrive -val csatlakoztatva indul és a pendrive filerendszerének gyökerében található egy `auto256k.tvc` nevű file, aminek a tartalma egy, a pendrive-on lévő könyvtár, akkor annak tartalma a gép induláskor automatikusan betöltődik.
- Az EXT CLI -ben az `LCDSR` parancs hatására az aktuális könyvtár tartalma betöltődik. Ha korábban volt más könyvtár gyorsítótárazva, az a elveszik, felülíródik.
- Programból van egy erre dedikált natív függvényhívás

A korábban említett `DS` parancs segítségével nem válthatunk slowRAM -ra, amíg nem töltöttünk be oda egy könyvtárat.

A fent említett `auto256k.tvc` file-t el is tudjuk készíttetni a kártyával, szintén az EXT CLI-ben, az `MKAUTO` paranccsal. Ekkor az aktuális könyvtár elérésével jön létre a file a csatlakoztatott pendrive gyökérkönyvtárában.
Figyelem: ha a file már létezett, akkor felülíródik.

Flash drive

A kártyára forrasztott fekete modulon található egy 16MB -os flash IC, ami gyakorlatilag ROM-ként funkcionál. Ebből 1MB-ot az MCU használ, itt tárolódik a kártya firmware -e, a maradék 15MB-on pedig egy csak olvasható fájlrendszer, játékokkal, demokkal, felhasználói programokkal. Kiválasztása a DS F paranccsal történik.

Ha a TVC-t úgy kapcsoljuk be, hogy a kártyában nincs pendrive, akkor ez lesz a kiválasztott drive induláskor.

A flash drive otthoni feltöltése

A flash alapvetően egy írható médium, csak az írása lassú, működés közben a kártyának túl sok dolga van ahhoz, hogy egy-egy blokk írásának az idejére leálljon. A TVC nem tudja írni a flash tartalmat, viszont arra van lehetőség, hogy otthon körülmények között újraírjuk ezt a meghajtót. Kb úgy kell elképzelni, hogy a összeállítjuk egy könyvtárból kiindulva a tartalmat - csak kisbetűs file- és könyvtárneveket használjunk! - , amit szeretnénk a flash drive -on látni (alkönyvtárakkal, mindenestül) és kiírjuk a modul flash chip-jére a megfelelő címtől kezdve.

Ehhez a következőket kell tennünk:

- github-ról (jelenleg: https://github.com/raspberrypi/pico-sdk-tools/releases/tag/v2.2.0-3) beszerezzük az ingyenesen letölthető picotool, PC-nk operációs rendszerének megfelelő változatát (windows alá nyilván a picotool-xxxx-win.zip file kell)
- Szintén github-ról beszerezzük a little-fs python programcsomagot (`python` -t telepítünk, `virtual environment`-et készítünk és `pip` segítségével felrakjuk a littlefs-python csomagot: `pip install littlefs-python`). Ezen lépések részletezése nem feladata ennek a dokumentumnak, tessék utánaolvasni, hogy mi micsoda.
- A `little-fs` tool -al elkészítjük az image file -t, amit használni szeretnénk:

  ```
  littlefs-python create -v --block-size 4096 --fs-size 15MB
  flash-root my-flash.bin
  ```

  A `flash-root` directory alatti file-ok fognak belekerülni a flash drive-ba, a block-size és az fs-size paraméterek kötelező értékek. Figyeljünk rá, h minden file neve kisbetűs legyen. Vagy egyenként megváltoztatjuk a file-ok neveit vagy megpatkoljuk a python programot. A választás a kedves felhasználóé.

- PC -t összekötjük a kártyával (amit kiveszünk a TVC-ből) a modul typeC csatlakozóján keresztól és boot módba helyezzük a kártyát (alul a BOOT és a RST gombokat lenyomjuk, a RST gombot engedjük fel először, majd a BOOT -ot. Egyetlen hüvelykujjal megvalósítható mutatvány, gyakorlat teszi a mestert..)

- A picotool-al feltöltjük az elkészített image file-t:

  ```
  picotool load my-flash.bin -t bin -o 0x10100000
  ```

  Nem egy villámgyors folyamat, de érdemes rá várni (kb 3-5 perc).

DSK Drive

A DSK Drive minden esetben egy létező .dsk file felmount-olása után lesz elérhető. A DSK drive írható és olvasható, azon file-okat, könyvtárakat tudunk létrehozni, átnevezni illetve törölni.

A .dsk image -nek - hasonlóan mint a pendrive -nak, FAT filerendszerrel kell rendelkeznie.

Lehetőség van a felcsatolt DSK drive lecsatolására (unmount) is. Ezt mindenképp meg kell tenni, ha egy másik .dsk file -t szeretnénk felmountolni.

Fel- és lecsatolni .dsk file -t több féle képpen lehet:

- az EXT CLI parancssorból
- TVC OS hívással
- TVC256k++ függvény hívással

LOAD

Programot a LOAD paranccsal lehet betölteni. A .cas kiterjesztést nem muszáj megadni - sőt, a TVC BASIC sajátossága miatt a stringet sem kell lezárni. Ez egy teljesen valid parancs:

```
LOAD "bobkiraly
```

Ennek hatására a kártya megkeresi az aktív drive-on a `bobkiraly.cas` nevű file-t a jelenlegi aktív könyvtárban és ha van ilyen, akkor az betöltődik. De egész filenevet is megadhatunk, teljes útvonallal:

```
LOAD "/games/bobkiraly.cas"
```

viszont mivel a kártya nem vált be a megadott könyvtárba, ezért pl utántöltős játékoknál ez nem jó gyakorlat, azok nem fognak így működni.

Lehetőség van a file név végén a * jelet használni, ekkor nem kell pl kiírni, hogy bobkiraly.cas, elég csak annyit, hogy

```
LOAD "bob*
```

Viszont itt vigyázni kell, mert ha az adott könyvtárban van másik file is, ami úgy kezdődik, hogy `bob`, de nem `.cas` kiterjesztésű, a betöltés elhasalhat (ha a katalógusban előrébb van, mint a `bobkiraly.cas`). Ha van másik `.cas` file, ami szintén `bob` -al kezdődik, akkor előfordulhat, hogy a LOAD azt a file-t találja meg először.

Ez a formátum csak könyvtárak nélkül működik, az aktív könyvtáron belül. A

```
LOAD "games/bob*"
```

parancs nem működik pendrive-ról (és felmountolt .DSK -ről. PSRAM és Flash drive esetében igen).

Ha a LOAD parancs után nem írunk semmit, akkor a kártya az aktuális drive-ról az aktuális könyvtárból beolvassa az első `.cas` file-t, amit csak talál. Ha talál.

Bekapcsolás során, pendrive nélkül a kártya megpróbálja inicializálni a flash drive -ot. Amennyiben sikerül neki, akkor a flash drive lesz az aktív. Egy ilyenkor kiadott LOAD parancs simán betölti a flash drive gyökerében található első (jó esetben egyetlen) programot, a launcher.cas -t. Ha van pendrive a kártyába dugva, akkor az USB drive lesz aktív bekapcsolás után.

SAVE

Programokat a SAVE paranccsal lehet kimenteni, de a flash drive nem írható. A SAVE parancs esetben, a filenévhez nem kell kiterjesztést írni, a kártya automatikusan hozzáilleszti a névhez a .cas kiterjesztést.

Lehetőségünk van már létező file-t felülírni, ekkor a filenevet a @ karakterrel kell kezdeni. Pl:

SAVE"@bobkiraly"

Könyvtárak

Mind a négy drive támogat alkönyvtárakat. Ezek bejárására, az aktív könyvtár megváltoztatására a EXT kiterjesztett parancsértelmezőben van lehetőségünk. Az ott található a CD, PWD, MKDIR és DELETE parancsokkal lehet aktív könyvtárat választani, lekérdezni az aktív könyvtárat, létrehozni és törölni könyvtárat. BASICből nincs lehetőség ezen műveletek végrehajtására, viszont van rá TVC OS fgv és a kártya natív fgv -ei is lehetőséget biztosítanak erre.

Programozás

IO PORTOK

A kártyát a belső regiszterein keresztül lehet programozni. Ehhez 2szer 3db IO port áll rendelkezésre. Először is meg kell határozni, hogy a kártya melyik SLOT-ba lett bedugva. Ezt vagy ránézésre tesszük meg: jobbról hányadik a slot, ahova a kártya be van dugva, vagy programból úgy lehet megtenni, hogy meg kell keresni a $0040, $0070, $00a0, $00d0 címekből melyik kezdetén van letárolva a db 6, "256k++" (06 32 35 36 6B 2B 2B) 7 byte -ból álló sorozat. Ez kijelöli a SLOT számot (0-3). A SLOT szám pedig kijelöli a BASE PORT címet: $10, $20, $30, $40

A BASE PORT meghatározásnak van még egy módja, keletkezett egy rendszerfüggvény, ami C regiszterben adja vissza, hogy mi a base port (rendszerhívás előtt B, AF és DE elmentendő).

Portok szerepe

A BASE PORT-hoz képest ez a portok szerepe:

+0: regiszter kijelölése. Ide írva a TVC256k++ számára kijelölünk egy regisztert, amit a későbbiekben írni, vagy onnan olvasni szeretnénk. Ez a kijelölés a regiszter írásakor/olvasásakor változhat, lásd lejjeb! Olvasásra a kártya visszaadja, hogy éppen melyik regiszter van kijelölve a következő regiszter hozzáférésre.

Megjegyzés: a TVC256k++ kártyának 256 címezhető regisztere van, ezekből sok nem használt.

+1: regiszter elérése. Ide írva a kijelölt regiszterbe írunk, innen olvasva a kijelölt regiszter értékét olvashatjuk vissza. A legtöbb esetben a regiszterbe írt érték visszaolvasható. **Figyelem!!** Vannak azonban olyan regiszterek is, amiknek az értéke írásra nem azonnal változik meg és olyan is, ahol a regiszter csak olvasható (a ráírt értéket a kártya figyelmen kívül hagyja).

+2: regiszter inkremens. Ide írva megadhatjuk, h egy regiszter hozzáférés után mennyivel változzon a kijelölt regiszter címe (nem az értéke!!). Ennek bekapcsoláskori értéke 1, azaz minden regiszter írás után a kijelölt regiszter eggyel feljebb ugrik, így akár egy regiszter kijelöléssel lehet több regiszter értéket is kiírni, visszaolvasni.

+4,+5,+6: ugyanolyan funkciójú portok, mint a +0,+1,+2, csak saját kijelölt regiszterrel, saját regiszter inkremenssel rendlekeznek. Ha megszakításban akarunk a TVC256k++ -ra írni/olvasni, érdemes ezt a második portkészletet használni és akkor a főprogram alap regiszter kijelölései nem sérülnek.

Megjegyzés: a +3, és a +7-től +15-ig tartó portokat a kártya nem használja

A portok használata nagyon hasonló, ahogyan a CRTC regisztereit érjük el, két fontos különbség van:

1: a portok duplázva vannak, így egy interruptból, alprogramból tudunk egy 2. regiszterkészletet címezni a már korábban kijelölt regiszter elrontása nélkül

2: a regiszterek elérését lehet automatikusan inkrementálni, pl ha tudjuk, h sprite-ok X koordinátáinak csak az alsó byte-ját akarjuk módosítani, akkor kettesével kell írni a regiszterekbe (hiszen spr0.xLow, spr0.xHi, etc... módon érhetjük el az X koordináta regisztereit a sprite-oknak).

REGISZTEREK I.

A kártya regiszterein keresztül lehet kommunikálni az MCU-ban futó programmal: hova rajzolja a sprite-ot, melyik lapot mutassa a P3 helyén, honnan vegye a karakterkészletet, stb... Ezek a regiszterek közül van

- ami csak olvasható

  Ezen regiszterekbe az írás nincs semmilyen hatással. Ilyen regiszterek pl a sprite-ok ütközés állapota (habár annak beolvasásakor az nullázódik és az esetleg interrupt

pedig ACK-olódik), az egér állapota (DX, DY szintén nullázódik), vagy a firmware verzió regiszterek.

- amik írhatóak is és az értéküket változtatva az azonnal érvényre jut
  Ilyen pl a memória lap kiválasztása vagy a sprite-ok fázisainak a kijelölése. A $90 alatti regiszterek mind ilyenek.
- aminek értékét változtatva nem azonnal jut érvényre
  A programozó dönti el, hogy megvárja-e vagy sem a regiszterbe írt adat feldolgozását. Az, hogy hogyan lehet tudni, h a regiszterbe írt adat feldolgozódott, az regiszterenként lehet más és más, de általában úgy lehet eldönteni, h visszaolvassuk a regiszter értékét és összevetjük az aktuális tartalmát a beleírt értékkel. Amíg nem az szerepel ott addig nem jutott érvényre a beírt érték. Ha BASIC -ből használjuk a kártyát, akkor a BASIC programok sebessége miatt általában (!) nem kell várakozni. Szép kivétel erre a fgv hívás regiszter ($C0). Ennek értéke akkor változik $FF -re, ha a kijelölt fgv végrehajtódott, addig a fgv kódja olvasható vissza a regiszterből.

RENDSZER FUNKCIÓK

Bekapcsoláskor a TVC256k++ kártya a CAS IO rendszer fgv-ek helyére fűzi be magát. A LOAD és a SAVE BASIC utasítások működnek. Utántöltős játékok, amennyiben a megfelelő könyvtárban vagyunk, működnek.

Az alábbi rendszer funkciók - file műveletek - (rst $30) kerültek implementálásra:

- char IO (read byte, write byte)
- blokk IO (read block to TVC RAM, write block from TVC RAM)
- open for read, for write
- close for read, for write
- verify

A fenti, standard file műveleteken túl további rendszerhívásokat is lehet megtenni, részletek egy későbbi fejezetben.

A LOAD parancsnál használható az üres string (simán csak LOAD). Ekkor a kártya az aktív könyvtárból az első .cas kiterjesztésű file-t fogja megpróbálni betölteni. Ha van ilyen.

A LOAD parancsnál filenevek végén használható a *, azaz ha nem akarjuk végig gépelni a teljes filenevet, egy * karakterrel helyettesíthetjük a maradék karakter sort. Ez az OPEN fgv sajátja, azaz ha nem .cas file-t talál először a katalógusban a kártya, akkor azt a TVC OS nem fogja tudni betölteni (START.BIN és START.CAS file -ok esetén kétesélyes, h elindul-e a program).

A SAVE parancsban (csak pendrive esetén!) nem használható sem az üres string, mint paraméter, sem a * jel. Amennyiben nem adunk .cas kiterjesztést a file-nak, a kártya azt automatikusan hozzáilleszti a filenévhez, és íráskor - mint minden .cas file létrehozásakor - egy 128 byte -os header is kerül a file elejére. Ezt a header -t egyébként olvasáskor a kártya nagyvonalúan átugorja.

Amennyiben javítani, felülírni szeretnénk egy file -t, akkor a SAVE esetén a filenév kezdődjön egy @ karakterrel és utána a filenév. Ha nem létezik a file, akkor a kártya

létrehozza azt. Ha létezik, akkor felülírja. Amennyiben nem @ jellel kezdődik a filenév mentés során és a file már létezik, akkor a mentés sikertelen lesz.

REGISZTEREK II.

Az alábbiakban felsorolom, h milyen regisztereket implementál a TVC256k++ kártya és azokat hogyan kell használni, értelmezni.

REG_SPRITE_X = $00

16 db sprite 16 bites (2 byte-os) X koordintája található a 00-1F regisztereken, alsó byte-felső byte formában, 0. sprite X koordintájától haladva a 15. sprite X koordinátájáig.
- írásra azonnal érvényre jut
- default: 0

REG_SPRITE_Y = $20 (32)

16 db sprite 16 bites (2 byte-os) Y koordintája található a 20-3F regisztereken, alsó byte-felső byte formában, 0. sprite Y koordintájától haladva a 15. sprite Y koordinátájáig.
- írásra azonnal érvényre jut
- default: 0

REG_SPRITE_COLOR = $40 (64)

16 db sprite szín regiszter, amennyiben a sprite 2színű üzemmódban van. 16 színű üzemmódban a regiszter értékét nem használja a kártya
- azonnal érvényre jut
- default: 0

REG_SPRITE_PHASE = $50 (80)

16 db sprite fázis regiszter. 2 színű üzemmódban a sprite fázisok adatai 64 byte-onkét, 16 színű módban pedig 256 byte-onként követik egymást a memóriában, a kijelölt sprite addresstől kezdődően.
- azonnal érvényre jut
- default: 0

Megjegyzés: a mikulássapi és a VIDEOTON betűk sprite -jai a memóriában megtalálhatóak induláskor

REG_SPRITE_FOREGROUND = $60 (96)

16 db sprite előtér- vagy háttérmód kiválasztása. Amennyiben az adott sorszámú sprite-hoz tartozó regiszterbe 0-t írunk, akkor a sprite a grafikai elemek mögött lesz kirajzolva (background), amennyiben 1 (nem 0), akkor előtérben lesz (foreground).
- azonnal érvényre jut. Ha nem 1-et írunk értékül, akkor az kis idő múltán változik 1-re.
- default: 1 (előtér)

Megjegyzés: minden TVC256k++ által kirajzolt grafikai elem a TVC alap képernyője *fölött* lesz látható

REG_SPRITE_ENABLE = $70 (112)

16 db sprite enable regisztere. 0 esetén a sprite nem jelenik meg a képernyőn, nem nulla (1) esetén pedig igen, amennyiben a koordinátái, kijelölt fázis adatai azt lehetővé teszik.
- azonnal érvényre jut. Ha nem 1-et írunk értékül, akkor az kis idő múltán változik 1-re.
- default: 0 (nincs bekapcsolva)

REG_SPRITE_COLORMODE = $80 (128)

16 db sprite colormode regisztere. 0 esetén a sprite 2 színű üzemmódban, 1 (nem 0) esetén pedig 16 színű üzemmódban lesz megjelenítve.
- azonnal érvényre jut. Ha nem 1-et írunk értékül, akkor az kis idő múltán változik 1-re.
- default: 1 (16 színű üzemmód)

REG_SPRITE_ENABLE_LOW = $90 (144)

Az alsó 8 sprite enable regisztere. Ha a bit értéke 0, akkor a sprite kikapcsolt (disabled), 1 esetben sprite bekapcsolt (enabled) állapotba kerül (ahol bit 0: sprite 0, bit 1: sprite 1, etc...). Egy sprite csak bekapcsolt állapotban látszódhat a képernyőn.
- írásra nem jut azonnal érvényre, akkor teljesült, amikor visszaolvasva ugyanaz az érték, mint amit a fejlesztő beleírt a regiszterbe.
A $70-$77 regiszterbe írások sem jelennek meg azonnal itt, de az oda írt értékek azonnal érvényre jutnak.
- default: 0

REG_SPRITE_ENABLE_HIGH = $91

Az felső 8 sprite enable regisztere. Ha a bit értéke 0, akkor a sprite kikapcsolt (disabled), 1 esetben sprite bekapcsolt (enabled) állapotba kerül (ahol bit 0: sprite 0, bit 1: sprite 1, etc...) Egy sprite csak bekapcsolt állapotban látszódhat a képernyőn.
- írásra nem jut azonnal érvényre, akkor teljesült, amikor visszaolvasva ugyanaz az érték, mint amit a fejlesztő beleírt a regiszterbe.
A $78-$7F regiszterbe írások sem jelennek meg azonnal itt, de az oda írt értékek azonnal érvényre jutnak.
- default: 0

REG_SPRITE_FOREGROUND_LOW = $92

Az alsó 8 sprite foreground regisztere. 0: background, 1: foreground, bit 0: sprite 0, bit 1: sprite 1, etc...
- írásra nem jut azonnal érvényre, akkor teljesült, amikor visszaolvasva ugyanaz az érték, mint amit a fejlesztő beleírt a regiszterbe.

A $60-$67 regiszterbe írások sem jelennek meg azonnal itt, de az oda írt értékek azonnal érvényre jutnak.
- default: 255

REG_SPRITE_FOREGROUND_HIGH = $93

A felső 8 sprite foreground regisztere. 0: background, 1: foreground, bit 0: sprite 8, bit 1: sprite 9, etc...
- írásra nem jut azonnal érvényre, akkor teljesült, amikor visszaolvasva ugyanaz az érték, mint amit a fejlesztő beleírt a regiszterbe.
A $68-$6f regiszter írások sem jelennek meg azonnal itt, de az oda írt értékek azonnal érvényre jutnak.
- default: 255

REG_SPRITE_COLORMODE_LOW = $94

Az alsó 8 sprite 16-szín üzemmód engedélyező regisztere. 0: 2c (2 szín), 1: 16c (16 szín), bit 0: sprite 0, bit 1: sprite 1, etc...
- írásra nem jut azonnal érvényre, akkor teljesült, amikor visszaolvasva ugyanaz az érték, mint amit a fejlesztő beleírt a regiszterbe.
A $80-$87 regiszter írások sem jelennek meg azonnal itt, de az oda írt értékek azonnal érvényre jutnak.
- default: 255

REG_SPRITE_COLORMODE_HIGH = $95

A felső 8 sprite 16-szín üzemmód engedélyező regisztere. 0: 2c (2 szín), 1: 16c (16 szín), bit 0: sprite 8, bit 1: sprite 9, etc...
- írásra nem jut azonnal érvényre, akkor teljesült, amikor visszaolvasva ugyanaz az érték, mint amit a fejlesztő beleírt a regiszterbe.
A $88-$8F regiszter írások sem jelennek meg azonnal itt, de az oda írt értékek azonnal érvényre jutnak.
- default: 255

REG_SPRITE_SP_COLLISION_LOW = $96

Sprite-sprite ütközés regiszter alsó 8 bitje. A sp-sp ütközésregiszterben azok a bitek 1-re vannak állítva, ahol az adott sprite egy másik sprite-tal átfedésben van (a nem-átlátszó pixelei). A teljes, 16 bites ütközésregiszter értéke nullázódik, amikor ezt a regisztert (a 16 bites ütközésregiszter alsó 8 bitjét) olvassuk.
Csak olvasható regiszter.
Olvasásra törlődik a regiszter tartalma.
Olvasásra az ütközés interrupt is törlődik (acknowledge).

REG_SPRITE_SP_COLLISION_HIGH = $97

Sprite-sprite ütközés regiszter felső 8 bitje.
Csak olvasható regiszter.
Ezen regiszter olvasásakor nincsenek mellékhatások, mint az alsó 8 bit olvasásakor.

REG_SPRITE_BG_COLLISION_LOW = $98

Háttér-sprite ütközés regiszter alsó 8 bitje. A bg-sp ütközésregiszterben azok a bitek 1-re vannak állítva, ahol az adott sprite a háttérrel átfedésben van (a nem-átlátszó pixelei). A teljes, 16 bites ütközésregiszter értéke nullázódik, amikor ezt a regisztert (a 16 bites ütközésregiszter alsó 8 bitjét) olvassuk. A sprite-ok background-foreground beállításai nem játszanak szerepet.
Csak olvasható regiszter.
Olvasásra törlődik a teljes 16 bites regiszter tartalma.
Olvasásra az ütközés interrupt is törlődik (acknowledge).

REG_SPRITE_BG_COLLISION_HIGH = $99

Háttér-sprite ütközés regiszter felső 8 bitje.
Csak olvasható regiszter.
Ezen regiszter olvasásakor nincsenek mellékhatások, mint az alsó 8 bit olvasásakor.

REG_SPRITE_SP_IRQMASK_LOW = $9A

Sprite-sprite interrupt kérés regiszter 0-7 sprite-okra vonatkozó bitmaszkja. Amennyiben a beállított biteknek megfelelő sprite egy másik sprite-al ütközik, úgy egy interrupt generálódik a képernyő kirajzolása után a Z80 processzorban, az alsó keret megkezdésekor. Az interrupt-ot törölni (acknowledge -elni) a sprite ütközésregiszter ($96) alsó byte-jának olvasával kell.

REG_SPRITE_SP_IRQMASK_HIGH = $9B

Sprite-sprite interrupt kérés regiszter felső 8 bitje - 8-15 sprite-okra vonatkozóan.

REG_SPRITE_BG_IRQMASK_LOW = $9C

Háttér-sprite interrupt kérés regiszter alsó 8 bitje, 0-7 sprite-okra vonatkozóan. Amennyiben a beállított biteknek megfelelő sprite a háttérrel ütközik, úgy egy interrupt generálódik a képernyő kirajzolása után a Z80 processzorban, az alsó keret megkezdésekor. Az interrupt-ot törölni (acknowledge -elni) a background ütközésregiszter ($98) alsó byte-jának olvasával kell.

REG_SPRITE_BG_IRQMASK_HIGH = $9D

Háttér-sprite interrupt kérés regiszter felső 8 bitje - 8-15 sprite-okra vonatkozóan.

REG_SPRITE_BASE_ADDR = $9E

A sprite fázisok bitmap képének a kezdetét jelöli ki a gyors RAM alsó 128kB -ján. Csak az alsó 3 bit számít, értéke 0, 1, ..., 7 lehet.
Hatására a fázisok 0x000000, 0x004000, ..., 0x01c000 címtől kezdődnek.
- írásra azonnal érvényre jut. A felső 5 bit az írás után kis késéssel nullázódik.
- default: 0

REG_SCREEN_VIDEOMODE = $A0 (160)

Az overlay képernyő grafikai elemeinek a video módját határozza meg (sprite-okra nincs hatással).
0: nincsenek grafikai elemek a képernyőn
1: 2 színű karakteres képernyő (1 szín + 1 átlátszó)
2: 16c karakteres képernyő (15 szín + 1 átlátszó)
3: bitmap üzemmód

- írásra azonnal érvényre jut, a felső 6 bit csak kis idő múlva törlődik.
- default: 0

REG_SCREEN_SCREEN_BASE_ADDR = $A1

A karakterek képernyő karakterkódjai az ún. screenRAM területen tárolódnak. 32x30 méretű byte-tömb, minden karaktersor 32 byte-ból áll.
Ezek a byte-ok jelölik ki, hogy a karakteres képernyő melyik karaktert jelenítse meg az adott karakter pozición.
Ez a regiszter a gyors RAM első 128kB-ján a screenRAM terület kezdetét jelöli ki. Az alsó 7 bit számít, a lehetséges értékek és a hozzájuk tartozó címek:

00 - 0x000000
01 - 0x000400
02 - 0x000800
...
7f - 0x01fc00

- írásra azonnal érvényre jut, a legfelső bit kis idő elteltével törlődik
- default: 1 (0x000400)

REG_SCREEN_SCREEN_COLOR_BASE_ADDR = $A2

A 2 színű karakteres képernyő karaktereinek a színét meghatározó blokk helyét jelöli ki. Minden byte alsó 4 bitje (irgb sorrendben) jelöli ki, h milyen színű legyen a megfelelő karakter a képernyőn.
Ez a regiszter a gyors RAM első 128kB-ján a screenRAM szín terület kezdetét jelöli ki. Az alsó 7 bit számít, a lehetséges értékek és a hozzájuk tartozó címek:

00 - 0x000000
01 - 0x000400
02 - 0x000800
...
7f - 0x01fc00

- írásra azonnal érvényre jut, a legfelső bit kis idő elteltével törlődik
- default: 2 (0x000800)

REG_SCREEN_FONT_BASE_ADDR = $A3

Ez a regiszter a gyors RAM első 128kB-ján a karakterképek területének a kezdetét jelöli ki (a 2 és a 16 színű fontokét egyaránt). Az alsó 6 bit számít, a lehetséges értékei:

00 - 0x000000
01 - 0x000800
02 - 0x001000
...
3f - 0x01f800

- írásra azonnal érvényre jut, a felső 2 bit kis idő elteltével törlődik
- default: 2 (0x001000)

Megjegyzés: A kártya a C=/plus4 magyarított karakterkészlettel a memóriában indul

REG_SCREEN_BITMAP_BASE_ADDR = $A4

Ez a regiszter a gyors RAM első 128kB-ján a bitmap területének a kezdetét jelöli ki. Az alsó 2 bit számít, a lehetséges értékei:

00 - 0x000000
01 - 0x008000
02 - 0x010000
03 - 0x018000

- írásra azonnal érvényre jut, a felső 6 bit kis idő elteltével törlődik
- default: 1 (0x8000)

Megjegyzés: a kártya a téli demóképpel a memóriában indul. Ez az a kép, amit a kártya fejlesztése közben generáltattam az AI -al. Hát... olyan is lett.

REG_SCREEN_SCROLL_X = $A5

A képernyő X irányú, balra eltolását teszi lehetővé. Alsó 3 bit határozza meg az eltolást, bit 7 pedig bekapcsolja a széles oldalsó keretet (1-1 karakter széles oszlop jelenik meg a keret mellett, eltakarva a sprite-okat és a grafikai elemeket is).
- írásra azonnal érvényre jut, kis idő elteltével törlődik a bit3-6 bitek tartalma.
- default: 0

REG_SCREEN_SCROLL_Y = $A6

A képernyő Y irányú, lefelé eltolását teszi lehetővé. Alsó 3 bit határozza meg az eltolást, bit 7 pedig bekapcsolja a széles alsó-felső keretet (1-1 karakter, azaz 8-8 pixel magas sor jelenik meg a keret alatt és fölött, eltakarva a sprite-okat és a grafikai elemeket is).
- írásra azonnal érvényre jut, kis idő elteltével törlődik a bit3-6 bitek tartalma.
- default: 0

REG_SCREEN_BORDER_COLOR = $A7

A szélesített keret színe alapból felveszi a $00 portra (TVC keretszín) írt szín értékét. Ezen azonban lehet változtatni is, ennek a regiszternek az írásával. A regiszter alsó 4 bitje számít, irgb sorrendben (azaz nem a TVC keret-szín $00 portjának megfelelően).
- írásra azonnal érvényre jut
- default: az utoljára írt keretszín

REG_SCREEN_Y = $A8

Annak a sornak a belső sorszámát tartalmazza, amit éppen számol az MCU. 21 és 255 közötti érték lehet csak (nincs 10 vagy 280). Függőleges képvisszafutás után a keret első soráig 21-est tartalmaz a regiszter, onnantól növekszik egészen 255-ig. Így a képernyőn a 233. sorban fogunk 255-öt olvasni, utána elkezd 0-tól növekedni egészen 6-ig.
- csak olvasható
- nem lehet a teljes képernyő sorait kiolvasni belőle, csak debug célokra használható

REG_SCREEN_MAXY = $A9

Kijelöli, hogy milyen magas legyen a megjelenítendő képernyő, képsorokban. Amennyiben a CRTC regiszterben a képernyőt összébb húzzuk, ezt a regisztert is be kell állítani ugyanarra a képernyő magasságra, különben szétesik a kép. Az MCU ennek a regiszternek a tartalmából tudja, h mennyi képernyő sort kell kiszámolnia egy képernyő esetén.
- írásra nem jut azonnal érvényre
- default: 240
- minimum érték 2, maximum érték 240

REG_MEMORY_P2 = $B0 (176)

Az U2 -es lap (gyors) lapozó regisztere. A kijelölt gyorsRAM kezdetének a felső 4 bitje. Értéke 00 és 0F között lehet, a felső biteket a kártya figyelmen kívül hagyja. Ez a regiszter által belapozott gyors RAMot a TVC a $8000-$BFFF között láthatja - amennyiben RAM, azaz U2 elérés van beállítva. Lehetséges értékek:
0 - 0x000000
1 - 0x004000
2 - 0x008000
3 - 0x00C000
4 - 0x010000
...
F - 0x03C000
$FF - nincs érvényben memória lap, ilyenkor a TVC a saját, belső RAMját látja az U2 tartományban - amennyiben van neki.
(256kB használható gyors RAM az hexadecimálisan 0x040000)
- írásra azonnal érvényre jut, felső 4 bit kis idő elteltével törlődik - amennyiben nem $FF a beállítás
- default: 4

REG_MEMORY_P3 = $B1

Az U3 -as lap (gyors) lapozó regisztere. A kijelölt gyorsRAM kezdetének a felső 4 bitje. Értéke 00 és 0F között lehet, a felső biteket a kártya figyelmen kívül hagyja. Ez a regiszter által belapozott gyors RAMot a TVC a $C000-$FFFF között láthatja - amennyiben RAM, azaz U3 elérés van beállítva. Lehetséges értékek:
0 - 0x000000
1 - 0x004000
2 - 0x008000
3 - 0x00C000
4 - 0x010000
...
F - 0x03C000
$FF írásakor nincs érvényben memória lap, ilyenkor a TVC a saját, belső RAMját látja az U3 tartományban - amennyiben van neki.
(256kB használható gyors RAM az 0x040000)
- írásra azonnal érvényre jut, felső 4 bit kis idő elteltével törlődik
- default: 5

REG_MEMORY_MAP_8M_P2_LOW = $B2

Az U2 -es lap lapozó regiszterének az alsó 8 bitje a lassú RAMból (PSRAM). 16kB -onként lehet lapokat kijelölni, 8 MB -on összen 512 lap van... (a fene egye meg, ez kicsin múlott..)
A regiszter írásakor a PPREG_MEMORY_MAP_8M_P2_HIGH tartalmával együtt kijelölődik egy 16kB-os lap a 8MB -os külső RAMban és a folyamat végén az elérhetővé válik a TVC számára a $8000-$BFFF címen - amennyiben U2 volt oda beállítva.

Belapozáskor, használat előtt a lassú RAMból a gyors RAMba kell másolni a 16kB -nyi adatot. Amennyiben korábban már lassú RAMból volt belapozva 16kB, akkor azt először a kártya visszamásolja a lassú RAMba és csak utána tudja bemásolni az új lap byte-jait. Lapozás után a REG_MEMORY_P2 ($B0) regiszter értéke $10 lesz.
- írásra késleltetve jut érvényre, amennyiben a terület használatra kész, a regiszter olvasásra a beleírt értéket adja vissza
- default: 0

REG_MEMORY_MAP_8M_P2_HIGH = $B3

Az U2 -es lap lapozó regiszterének a legmagasabb helyiértékű bitje. Írásra nem indul meg a belapozási folyamat. A regiszter felső 7 bitje a belapozás során törlődik.

REG_MEMORY_MAP_8M_P3_LOW = $B4

Az U3 -as lap lapozó regiszterének az alsó 8 bitje a lassú RAMból (PSRAM). 16kB -onként lehet lapokat kijelölni, 8 MB -on összen 512 lap van...
A regiszter írásakor a PPREG_MEMORY_MAP_8M_P3_HIGH tartalmával együtt kijelölődik egy 16kB-os lap a 8MB -os külső RAMban.
Belapozáskor, használat előtt a lassú RAMból a gyors RAMba kell másolni a 16kB -nyi adatot. Amennyiben korábban már lassú RAMból volt belapozva 16kB, akkor azt először a kártya visszamásolja a lassú RAMba és csak utána tudja bemásolni az új lap byte-jait. Lapozás után a REG_MEMORY_P3 ($B1) regiszter értéke $11 lesz.
- írásra késleltetve jut érvényre, amennyiben a terület használatra kész, a regiszter olvasásra a beleírt értéket adja vissza
- default: 0

REG_MEMORY_MAP_8M_P3_HIGH = $B5

Az U3 -es lap lapozó regiszterének a legmagasabb helyiértékű bitje. Írásra nem indul meg a belapozási folyamat. A regiszter felső 7 bitje a belapozás során törlődik.

REG_MEMORY_PSRAM_SIZE_IN_MB = $B6

A külső, lassú RAM (PSRAM) mérete, MB -ban. Normál esetben ez 8. Csak olvasható regiszter.

REG_MEMORY_ROM_PAGE = $B7

Az IO ROM lapozó regisztere. Ez határozza meg, hogy a TVC számára látható ROM terület első 6kB -jára melyik 6kB-os blokk legyen bemásolva. A lehetséges értékek 0 vagy 1. Ez lassú művelet (hasonlóan, mint a PSRAM belapozása). A másolás befejeztét az jelzi, hogy a regiszter olvasásra a beírt értéket adja vissza - azaz a regiszter értéke csak a másolás befejeztével állítódik be. 1-nél nagyobb érték beírását a kártya figyelmen kívül hagyja.

REG_FUNCTION_EXECUTE = $C0 (192)

Egy beépített függvény végrehajtása. A regiszterbe kell írni a fgv kódját, s amennyiben a fgv végrehajtódott (akár jól, akár hibakóddal), úgy a regiszter olvasásra 255 -öt ad vissza. A végrehajtás nem azonnali, a fgv hívás eredményei - amennyiben van - csak a fgv végrehajtása után olvasható ki a memóriából. Meg kell különböztetnünk a fgv visszatérési értékét (ami 0, ha a fgv rendben lefutott, vagy valami, a fgv-re jellemző hibakód), illetve a fgv kimenő paramétereit egymástól. Ez utóbbi lehet pl az aktuális toll színe, vagy az aktuális kurzor pozíció, esetleg a fileból kiolvasott adatblokk.
A fgv -ek részletes ismertetése egy későbbi fejezetben találhatóak.
A fgv-ek a paramétereiket a `reg_function_param_start` által kijelölt memória területről olvassák és a kimenő paramétereiket is itt helyezik el.
A fgv -ek visszatérési értékét a `reg_function_result` regiszterből lehet kiolvasni.

Veszély: figyelmetlenül felparaméterezett fgv hívás könnyen tud rendszerösszeomlást okozni (Hardware Fault). Ezt a regisztert programjainkban körültekintően használjuk!

REG_FUNCTION_PARAM_START = $C1

A függvény paramétereit mindenképpen belapozott (gyors vagy lassú) RAMban lehet csak átadni. Ezzel a regiszterrel jelölhetjük ki, hogy hol kezdődjön a paraméter terület. 128 byte-os lépésekben jelölhetjük ki az aktuálisan belapozott U2 vagy a U3 területről.

0-127 : a belapozott U2 területről veszi a paramétereket. Kezdőcím a TVC-ben: $8000 + PS * 128 (ahol a PS a param start regiszter értéke)
128-255 : a belapozott U3 területről veszi a paramétereket. Kezdőcím a TVC-ben: $C000 + (PS-128) * 128 (ahol a PS a param start regiszter értéke)

REG_FUNCTION_RESULT = $C2

A fgv végrehajtás után innen olvasható ki a fgv visszatérési értéke - ha van neki olyan. A hibát minden esetben a nem nulla hibakóddal jelzi a kártya.
Csak olvasható regiszter

REG_FW_VERSION_MAJOR = $C3

A kártya firmware verziójának a fő száma.
Csak olvasható regiszter.

REG_FW_VERSION_MINOR = $C4

A kártya firmware verziójának a minor száma.
Csak olvasható regiszter.

REG_DRIVE_SELECT = $C5

Kijelöli, hogy a TVC file műveletei honnan olvassák a file-okat. PSRAM és DSK drive -okat csak akkor szabad kijelölni, ha előtte létrehoztuk/felmountoltuk azokat. A regiszter beállítása hibát nem fog jelezni, minden művelet viszont igen.

0 - USB (írható/olvasható)
1 - PSRAM (írható/olvasható, kisbetűs filenevek)
2 - Flash (csak olvasható, kisbetűs filenevek)
3 - DSK (abban az esetben, ha olvasható filerendszerről mountoltuk be, akkor írható is).

Ez az egyetlen ún. memory mapped (kb. memóriához rendelt) regiszter. Ez a regiszter történeti okokból az IO ROM tartomány $D810 -es címén is elérhető (ahol valójában emulált RAM található).

REG_FUNCTION_MULTI_EXECUTE = $C6

Több függvény hívása egymás után. Az egyes fgv-ek csak akkor hajtódnak végre, ha a listában az előtte álló fgv hívása sikeres volt, azaz 0-val tért vissza a meghívott fgv.

Veszély: figyelmetlenül felparaméterezett fgv hívás könnyen tud rendszerösszeomlást okozni (Hardware Fault). Ezt a regisztert programjainkban körültekintően használjuk!

INPUT:

A regiszterbe kell írni, hogy mennyi fgv hívást fűztünk össze (`num_of_function_calls`).

Az alább ismertetett paramétereket a `REG_FUNCTION_PARAM_START` regiszter által kijelölt területről olvassa be a kártya. Amikor a kártya végzett a kijelölt funkciók végrejtásával, a regiszter tartalma $FF lesz, olvasásra ez az érték fog visszajönni.

INPUT:

A paraméterezés `num_of_function_calls` -szor ismétlődik:

- function_code (1): a soron következő, meghívandó fgv kódja (lásd a A REG_FUNCTION_EXECUTE függvény hívások fejezetet)
- Params (n): itt pedig a meghívandó fgv paraméterei, n db byte, ahol n az adott fgv paraméter blokkjának a hossza (lehet 0 is). N-et nem kell megadni sehol sem, ezt a kártya tudja, pontosabban ki tudja számolni.

OUTPUT:

- Ha volt hibás fgv hívás, akkor az adott fgv hibakódja kerül a `REG_FUNCTION_PARAM_START` regiszter által kijelölt terület 0. bájtjára, a REG_FUNCTION_RESULT pedig az elhasalt fgv sorszámát fogja tartalmazni
- A fgv hívások kimenő paraméterei felülírhatják a paraméterlistát, de csak a fgv a saját területétől ír(-hat).

retVal (REG_FUNCTION_RESULT):

- 0: ha minden kijelölt fgv -t sikeresen végre lehetett hajtani
- k: a k. (1-től kezdve a számozást) fgv -nek a visszatérési értéke nem nulla volt (az utána következő fgv -eket már nem hajtotta végre a kártya). A visszatérési érték (hibakód) csak ebben az esetben kerül bele a kimeneti területre.
- $80+k: a k. (1-tól kezdve a számozást) fgv kódja nem ismert.

REG_FUNCTION_BITMAP_BASE = $C7

Kiválasztható, hogy a bitmap kezelő rutinok (`clear_bitmap_screen, set_dot_color, get_dot_color, draw_line, draw_rectangle, fill_rectangle, draw_multi_line, scanline_flood_fill, draw_ellipse, fill_ellipse, copy_image_block, copy_sub_image`) milyen címen (célcímmel) rajzoljanak. Ennek a regiszternek a segítségével lehet képeket egy háttérbufferben megalkotni. A háttérbuffer egy megjeleníthető 32kB -os blokk a gyorsRAMban.

A regiszter írható és olvasható.

Lehetséges értékek:
- $00, $01, $02 és $03, amik a $000000, $008000, ... $018000 címeket jelöli ki a kártya fastRAM-ban.
- $FF: ekkor a célcím az éppen megjelenítendő, REG_SCREEN_BITMAP_BASE_ADDR regiszter által kijelölt bitmap terület.

REG_DRIVE_PSRAM_VALID = $C8

Csak olvasható regiszter, megmutatja, hogy van-e a kártyán jelenleg formázott, inicializált PSRAM drive. Írásra nem történik semmi, olvasásra:

0 - nincs

Egyéb: van

REG_DRIVE_FLASH_VALID = $C9

Csak olvasható regiszter, megmutatja, hogy van-e a kártyán jelenleg formázott, inicializált FLASH drive. Írásra nem történik semmi, olvasásra:

0 - nincs

Egyéb: van

REG_DRIVE_DSK_VALID = $CA

Csak olvasható regiszter, megmutatja, hogy van-e a kártyán jelenleg felmount-olt, valid DSK drive. Írásra nem történik semmi, olvasásra:

0 - nincs

Egyéb: van

REG_USB_INIT = $D0 (208)

Ez a regiszter jelzi, ha a kártya valamilyen USB eszközt érzékel.
$00: nincs USB eszköz
$01: USB egér
$02: pendrive
Csak olvasható regiszter
Megjegyzés: a kártya nem kezel USB HUB -ot, egyszerre csak egy eszköz lehet csatlakoztatva.

REG_USB_MOUSE_BUTTONS = $D1

Az USB egér lenyomott gombjai. 1 = bal, 2 = középső, 4 = jobb egérgomb. Ha több gomb van egyszerre lenyomva, akkor az értékek logikai VAGY eredménye olvasható a regiszterből.
Csak olvasható regiszter

REG_USB_MOUSE_DX = $D2

Az utolsó olvasás óta történt x irányú egér elmozdulás.
Csak olvasható regiszter

REG_USB_MOUSE_DY = $D3

Az utolsó olvasás óta történt y irányú egér elmozdulás.
Csak olvasható regiszter

REG_USB_MOUSE_DW = $D4

Nem implementált. Azt tapasztaltam, hogy 0 és 1 értékek jönnek vissza a driverből, -1 sosem. Inkább nem erőltettem.

REG_USB_MOUSE_SPEED = $D5

Az egér mozgás sebessége, szorzó. Felső 3 bit: egész rész, alsó 5 bit: tört rész.
Írásra az egér sebessége állítódik.
Ez az egyetlen regiszter, melynek a tartalma ki- és bekapcsolások között is megőrződik, a regiszterbe írt érték minden a flashben tárolódik.
Amire oda kell figyelni, az érték nem overwrite módban íródik bele a flash -be, hanem egy 4kB -os bufferbe kerül bele. A kártya csak induláskor tudja teljesen törölni a buffert, és csak akkor törli, ha a bufferből már csak 256 használható byte van (azaz 256-szor minden esetben biztonságosan megváltoztathatjuk az egér sebességét, de legfeljebb 4094-szer - legalábbis egy be- és kikapcsolás között)
- default: $10 (0.5x szorzó)

REG_USB_MSC_CMD = $D6

File műveletek a kártya IO ROM rutinjai részére (paramétereket az IO ROM terület írható részén ad át a kártya). Ugyanazok a file műveletek találhatóak itt is, mint a REG_FUNCTION_EXECUTE esetén a 128-nál nagyobb függvények, csak itt a paraméterátadás minden esetben az IO ROM terület felső 2kB részén, az IO RAM-ban zajlik. Használhatóságát erősen korlátozza, hogy az IO ROM-nak belapozva kell lennie, ezért itt nem is dokumentálom. Akit mélyebben érdekel a dolog, a forrást lapozgassa.

REG_SID_BASE = $E0 (224)

vSID regiszterek. A kártya gyakorlatilag tartalmaz egy SID chip emulációt. Majdnem. Filter regiszterek nincsenek implementálva/használva. A SID regiszter leírásokért fordulj bizalommal az internethez.
Figyelem: minden regiszter írható és olvasható a vSID -en (nem úgy, mint a SID regiszteren).

TVC OS rendszer fgv hívások

A kártya IO ROMjában a TVC CAS rutinok lettek felüldefiniálva és azokon túl még pár extra rutint szolgáltat az IO ROM.

Ezek a rutinok nyilván csak akkor fognak helyesen működni, ha a TVC256k++ kártya van befűzve a CAS rutinokhoz (értsd: nincs kártyától jobbra egyéb file IO kártya bedugva a gépbe).

A hibakódok minden esetben az A regiszterben érkeznek vissza. Extra hibakód olvasható ki még a kártyához tartozó RAM Base ($0040, $0070, ...) + 9 byte-ról. Ezeket a hibakódokat vagy a fatfs vagy a little_fs C library adja vissza, taglalásukba itt nem bocsátkozom.
A fatfs lib által kezelt file rendszerekre (fat16, fat32, exFat) jellemző, hogy minden 12 karakternél hosszabb névvel rendelkező file elérhető egy rövidített (12 karakteres) filenévvel is. Ez csak az USB drive-on elérhető funkció. A PSRAM és a flash drive -okon listázni lehet rövid nevekkel, de file-okat megnyitni nem.
A kártya egyszerre több file-t enged megnyitni viszont a TVC OS file műveletek egyszerre csak egy nyitott file-t tudnak kezelni. Ha szükség van több file egyidejű megnyitására (max 4 file), akkor mindenképpen a REG_FUNCTION_EXECUTE alatti fileműveteket kell használni.
A TVC OS függvényeit gépi kódból lehet használni, a RST $30 utasítás segítségével. A meghívandó fgv kódja az utasítás utáni byte a memóriában, míg a paraméterei a B, C, D és E regiszterek tartalmazhatják. Például:

```
LD DE, $3000
RST $30
DB $D6 ; get_pwd
OR A
JR nz, HIBA
...
```

A szokásos CHRIN, CHROUT, OPEN (CREATE, OPEN), CLOSE, BLKIN, BLKOUT és VERIFY rutinokon túl - melyek az aktuális drive aktuális könyvtárára vonatkozó fileműveletek - az alábbi fgv-ek lettek implementálva:

GET_PWD - $D6

Visszaadja az aktuális munkakönytárat.

INPUT:

DE: pointer egy legalább 256 byte hosszú bufferre

OUTPUT:

A:

0 - OK, a terület, ahova DE mutat a full directory nevét tartalmazza (/ jellel a végén). TVC string, azaz az első byte a string hosszát, utána pedig max 255 karaktert tartalmaz.

$FE - nincs pendrive, pedig USB drive lett kiválasztva

$EE - IO hiba

CH_DIR - $56

Megváltoztatja az aktuális könyvtárat a kiválasztott drive-on. PSRAM -on ez a fgv hibával tér vissza.

INPUT:

DE: pointer egy TVC stringre, ami lehet relatív, vagy abszolút útvonal

OUTPUT:

A:

0 - OK, a könyvtárváltás sikeres
$ED - a könyvtárváltás nem sikerült
$FE - nincs pendrive, pedig USB drive lett kiválasztva

OPEN_DIR - $57/$D7

Egy könyvtár listázása minden esetben a könyvtár megnyitásával kezdődik, majd a könvytár bejegyzések olvasása következik és végül a könyvtárat le kell zárni.

Az OPEN_DIR rendszer fgv mindig az aktuális könyvtárat (.) listázza, nincs input paraméter.

OUTPUT:

$00 - OK, a könyvtár olvasásra kész
$F0 - IO hiba
$EF - nincs pendrive, pedig USB drive lett kiválasztva

READ_DIR - $D8

Könyvtár bejegyzés olvasás. Ezt a fgv -t csak egy hibátlan OPEN_DIR után szabad hívni.
INPUT:
DE: pointer egy legalább 274 byte hosszú bufferre, ahova a fgv hívás másolja be a könyvtár bejegyzés adatait
OUTPUT:
A:
$00 - OK, a könyvtár bejegyzés struktúra valódi adatokkal van feltöltve
$F1 - a könyvtár bejegyzést nem sikerült olvasni
$FE - nincs pendrive, pedig USB drive lett kiválasztva

A visszaadott könyvtár struktúra a következőkből áll:
- 256 byte, TVC sztring. Ez a file/könyvtár neve.
- 1 byte, attribútum. Ez csak USB pendrive esetén teljes. SlowRAM (PSRAM) drive esetben mindig $00, flash drive esetben pedig $00 (file) vagy $10 (könyvtár).USB drive esetben a lehetséges értékek:
- $01 - read only
- $02 - hidden
- $04 - system
- $10 - directory
- $20 - archive
- 4 byte, file mérete. Könyvtár esetén nincs feltöltve. Flash drive esetben max 2^31-1. Fatfs-en 4GB -nál (exFat) nagyobb file -ok esetén mindig $FFFFFFFF. Alacsonytól a magas helyiértékig tárolódnak a byte-ok, azaz egy 09012345h hosszú file mérete így tárolódik a memóriában: 45 23 01 09
- 13 byte, rövid filenév, TVC string. Csak abban az esetben töltődik ki, ha a file neve hosszabb, mint 12 karakter.
A könyvtár listázás végét az jelzi, ha 0 hosszúságú filenév érkezik vissza a fgv hívásból (azaz a struktúra első byte -ja 0). Ez után már lezárnunk kell a könyvtárat a CLOSE_DIR hívással.

CLOSE_DIR - $58

Lezárja a korábban listázásra megnyitott könyvtárat. OPEN_DIR után minden esetben meg kell hívni.

A:

0 - OK, a könyvtár lezárása sikeres
Egyéb: a lezárás sikertelen

MK_DIR - $59/$D9

Létrehoz egy könyvtárat. Ez a hívás csak USB, PSRAM és DSK drive esetben implementált. A Flash drive csak olvasható drive.

INPUT:

DE: pointer egy TVC sztringre, ami tartalmazhat lehet relatív, vagy abszolút útvonalat

OUTPUT:

A:

0 - OK, a könyvtár létrehozása sikeres
$EF - a könyvtár létrehozása nem sikerült
$FE - nincs pendrive, pedig USB drive lett kiválasztva

DELETE - $5A/$DA

Egy file, vagy üres könyvtár törlése. Ez a hívás csak USB, PSRAM és DSK drive esetben implementált. A Flash drive csak olvasható drive.

INPUT:

DE: pointer egy TVC sztringre, ami tartalmazhat relatív, vagy abszolút útvonalat, filenevet.

OUTPUT:

A:

0 - OK, a törlés sikeres
$EF - a törlés nem sikerült
$FE - nincs pendrive, pedig USB drive lett kiválasztva

RENAME - $5B/$DB

Egy file, vagy könyvtár átnevezése. Ez a hívás csak USB, PSRAM és DSK drive esetben implementált. A Flash drive csak olvasható drive.

INPUT:

DE: pointer két egymást követő TVC stringre. Az első string tartalmazza azt, hogy miről, a második sztring pedig a mire file vagy könyvtár neveket tartalmazza

OUTPUT:

A:

0 - OK, a file/könyvtár átnevezése sikeres
$EF - az átnevezés nem sikerült
$FE - nincs pendrive, pedig USB drive lett kiválasztva

SEEK_FILE - $5C/$DC

Megnyitott file -on belüli aktuális pozíció beállítása.

INPUT:

DE: pointer egy négy byte-os struktúrára, ahova a filepointert állítani kell.

OUTPUT:

A:

$00 - OK, a seek sikeres
$E0 - nincs megnyitott file
$E1 - a seek nem sikerült
$FE - nincs pendrive, pedig USB drive lett kiválasztva

GET_IOBASE - $5D/$DD

C regiszterben visszaadja a kártya IO base címét ($10, $20, ...), az A regisztert pedig $00 -ra állítja.

GET_MEMBASE - $5E/$DE

C regiszterben visszaadja a kártya mem base címének alsó byte-ját ($40, $70, ...), az A regisztert pedig $00 -ra állítja.

MOUNT DSK - $5F

Felcsatol egy megjelölt .dsk fájlt, ezáltal létrehozva a DSK drive -ot. A .dsk file nevének címét a DE regiszter tartalmazza.

INPUT:

DE: pointer egy TVC sztringre, ami a .dsk file nevét tartalmazza

OUTPUT:

A:

0 - OK, a felcsatolás sikeres
$EF - a felcsatolás nem sikerült

UNMOUNT DSK - $DF

Lecsatolja a rendszerből a DSK drive -ot és ha DSK Drive volt aktív, akkor az aktív drive -ot beállítja arra a drive-ra, ahol a felcsatolt .dsk file volt.

INPUT: -

OUTPUT:

A:

0 - OK, a lecsatolás sikeres
$EF - a lecsatolás nem sikerült

A REG_FUNCTION_EXECUTE függvény hívások

A TVC256k++ kártya egy nagyon sokoldalú bővítő kártya. A regiszterei segítségével a TVC grafikai és hang képességei bővültek drasztikusan, de van egy funkció, amiről eddig nem volt részletesen szó: a kártya rendelkezik egy saját függvénykönyvtárral, amik memória kezelést, grafikai- és fájlműveleteket gyorsítja, bővíti ki. Nagyon fontos, hogy a fgv-ek csak belapozott külső (gyors vagy lassú) memória területekről tudják fogadni a paramétereiket).

Érdemes még azt figyelembe venni, hogy a függvény feldolgozása mindig késleltetett. Úgy kell elképzelni, h egy queue -ba bekerül a fgv kódja és amikor a feldolgozó MCU mag ráér, akkor elkezdi végrehajtani a kijelölt műveletet. Két fgv -t nem szabad egymásra 'ráindítani', hiszen a paramétereik, visszatérési értékeik nincsenek queue -ban, azok a memóriában összekeveredhetnek...

Hogyan is lehet meghívni egy függvényt a kártyán?

Param start

Az első lépésként be kell állítani, hogy honnan fogadják a fgv -ek a paramétereket, erre szolgál a REG_FUNCTION_PARAM_START regiszter. 0-127 értékek esetén a paramétereket az MCU a $8000-$BFFF címre belapozott RAM területről kapja (128 byte -os lépésekben állítható). 128-255 közötti érték beállítása esetén a fgv-ek a $C000-$FFFF címek valamelyikéről fogják várni a paramétereiket (szintén 128 byte-os lépésekben).

Vannak olyan fgv-ek, amiknek nincs szüksége paraméterre (pl: clr_text_screen), viszont a fgv hívások egyik feltétele, hogy a kijelölt paraméter terület be legyen lapozva a TVC RAM területére: ha nincs külső RAM belapozva, akkor a clr_text_screen is hibával fog visszatérni.

Execute és result

A függvény végrehajtás úgy zajlik, hogy a kiválasztott fgv-t felparaméterezzük a korábban kijelölt memóriaterületen, majd a fgv kódját beírjuk a REG_FUNCTION_EXECUTE regiszterbe. Ennek hatására (kicsivel később) elindul a fgv végrehajtása.

Amig a fgv fut az MCU -n belül, addig ennek a regiszternek az értéke (olvasásra) a fgv kódja lesz, amit beírtunk oda. Amint a fgv elkészült, a regiszter értéke $FF -re változik és a fgv visszatérési értéke kiolvasható lesz a REG_FUNCTION_RESULT regiszterből. A visszatérési érték felső bitje ($80) jelzi, h történt-e hiba vagy sem.

A visszatérési érték (retVal) nem összekeverendő a kimenő értékekkel (OUTPUT). Az előbbiben csak hibakódokat (vagy OK státuszt) ad vissza a fgv, az utóbbi pedig a fgv valódi kimenő értékeit tartalmazza (active color, open file ID, stb...)

Általános rutinok

Az alábbiakban felsorolom, hogy milyen függvények és milyen kódokkal lettek implementálva. Az egyes paraméter nevek mögött zárójelben a paraméter hosszát jelölöm, byte-okban mérve. Ha egy számot várunk paraméterül, akkor az mind little-endian sorrendben van, azaz az első byte a legkisebb helyiérték.

Clear_text_screen ($00)

Ez a rutin letörli ($20 -al tölti fel) a karakter képernyő screen RAM területét, kitölti a screen ColorRAM-ot az aktuális font színnel és beállítja a kurzorpozíciókat a bal felső sarokba (0,0). A képernyőtörlést a REG_SCREEN_MAXY regiszter értékétől függően végzi el, csak a látható tartományt törli le.

Nem vár paramétert és mindig 0-val tér vissza.

Clear_bitmap_screen ($01)

Ez a rutin letörli ($88-al tölti fel) a bitmap képernyő RAM területét (fixen 32768 byte -ot). Nem vár paramétert és mindig 0-val tér vissza. Vegyük észre, hogy a $88 az két egymás melletti pixelt állít átlátszóra.

Print_character_screen_code ($02)

Beállít egy képernyő kódot (nem ascii karaktert, hanem az aktuális fontkészletben, az adott képernyő kód által kijelölt karaktert) az aktuális kurzorpozícióba, majd növeli az X pozíciót. Következő sorba ugratja a kurzort, ha kell. Sort emel, ha kell.

Pl. a TVC256k++ kártyában található default font-ban a 0 -ás képernyő kód az @ jel, az 1 -es pedig az 'A' betű, stb.

INPUT:
- screen_code (1): a képernyő kód, amit meg kell jeleníteni

Mindig 0-val tér vissza.

Print_hex_byte ($03)

Kiír a képernyőre egy byte -ot hexadecimális formában, az aktuális kurzor pozíciótól, az aktuális karakter színnel (2 színű karakter módban). A C/+4 .es fontkészlet szerint működik.

INPUT:
- byte_value (1): a hex formában kiírandó byte

Mindig 0-val tér vissza.

Print_hex_word ($04)

Kiír a képernyőre egy word -ot (2 byte -os értéket) hexadecimális formában, az aktuális kurzorpozíciótól, az aktuális karakter színnel (2 színű módban). A C/+4 .es fontkészlet szerint működik.

INPUT:
- word_value(2): a hex formában kiírandó word.

Mindig 0-val tér vissza.

Print_string_screen_code ($05)

Egy képernyő kódból álló karakterláncot ír a képernyőre. A karakterlánc $00 -val kell végződjön.

INPUT:
- string(-): képernyő kódokból álló, null-terminált string, hossza gyakorlatilag tetszőleges.

Mindig 0-val tér vissza.

Print_string_ascii ($06)

Végrehajt egy esetlen konverziót ascii -ról a képernyőkódokra és egy ‘rendes’, fordító által befordított karakter stringet próbál a képernyőre kiírni a default (C+4) fontkészlettel.

Speciális karakterek:

$04, $13, $05, $18 - kurzor mozgatása a TVC editor parancsok szerint (jobbra, balra, föl, le). Képernyő széleken nincs scrollozás

$0C - képernyőtörlés

$0D - soremelés, scrollozás, kurzor a sor elejére ugrik (DOS like)

$20-$3F, a-z, A-Z mint ASCII

$80-$87: Á, É, Í, Ó, Ö, Ő, Ú, Ü, Ű (TVC szerint)

$90-$97: á, é, í, ó, ö, ő, ú, ü, ű (TVC szerint)

$A0-$DF: a fontkészlet speciális karakterei (leképezve $40-$7F közötti karakterekre)

$F0-$FF - aktuális karakterszín (az alsó 4 bit).

INPUT:
- string(-): fenti, ascii jellegű kódokból álló, null-terminált string, hossza gyakorlatilag tetszőleges.

Mindig 0-val tér vissza.

Get_text_color ($07)

A paraméterek első helyére beállítja az aktuális karakterszínt.

OUTPUT:
- color(1): az aktuális karakter szín

Paramétert nem vár, mindig nullával tér vissza.

Set_text_color ($08)

A paraméterül kapott byte -ot beállítja az aktuális színnek.

INPUT:
- color(1): a beállítandó karakter szín (`---irgb` formátumban)

Mindig nullával tér vissza.

Get_XY ($09)

Visszaadja az aktuális karakter pozíciókat.

OUTPUT:
- X (1): az aktuálisan kijelölt oszlop (0-31)
- Y (1): az aktuálisan kijelölt sor (0-23)

Mindig 0-val tér vissza.

Set_XY ($0A)

Beállítja az aktuális kurzor pozíciókat. A paraméterek első byte -ja jelenti az X, a második byte pedig az Y pozíciót.

INPUT:
- X (1): a karakteres képernyőn képernyőn az x koordináta. 255 esetén a fgv hívás során nem változik az aktuális x kurzorpozíció. Egyéb esetben 0-31 közötti értékek. Ha 32 vagy

annál nagyobb (de <255) értéket adunk meg, akkor 31 lesz az alapértelmezett érték.
- Y (1): a karakteres képernyőn az y koordináta. 255 esetén a fgv hívás során nem változik az aktuális y kurzorpozíció. Egyéb esetben 0-29 közötti értékek. Ha 29 vagy annál nagyobb (de <255) értéket adunk meg, akkor 29 lesz az alapértelmezett érték.

retVal:
Ez a fgv mindig 0-val tér vissza.

Memory_move_short ($0B)

Kevesebb, mint 16kB -nyi memóriát másol egy kijelölt blokkon belül, figyelembe véve az területek átlapolódását.

INPUT:
- memory_block (1): a 16kB -os blokk, ahol másolni kell (0-17). >15 esetben arra a felhasználónak kell figyelni, hogy valóban legyen belapozott lassú RAM
- destination (2): a blokkon belüli célcím
- source (2): a blokkon belüli forrás cím
- length(2): a blokk hossza, amit másolni kell

retVal:
- 0: sikeres másolás, vagy ha a source és destination ugyanaz, esetleg a length 0
- $81: memory_block nagyobb mint 17 (A 16. és a 17. lapok a lassú RAM területe)
- $82: a megadott paraméterekkel a másolás kilógna a blokkból

Memory_move_full ($0C)

A teljes gyors RAM-on belül másolunk egy memória blokkot, kezelve az átlapolódást.

INPUT:
- destination (3): a cél cím
- source (3): a forrás cím
- length (3): a másolandó blokk mérete

retVal:
- 0: sikeres másolás
- $81: a megadott paraméterekkel kilógunk a 256kB -os (szigorúan csak a gyors RAM) belüli területről

Memory_move_from_slow ($0D)

Adat másolás a lassú RAM-ból a gyors RAM-ba. Ehhez nem kell belapozni sehova semmit.

INPUT:
- destination (3): a cél cím a 256k méretű gyors RAM-on belül ($000000 - $100000)
- source (3): a forrás cím a 8MB -os lassú RAM-on belül ($000000 - $800000)
- length (3): a másolandó blokk mérete (<$100000)

retVal:
- 0: ha a másolás rendben megtörtént
- $81: ha vagy a lassú, vagy a gyors RAMból kilógunk a másolás közben

Memory_move_to_slow ($0E)

Adat másolása a lassú RAM-ba a gyors RAM-ból. Figyelem, ez a művelet tönkreteheti a PSRAM-ba bemásolt gyorsító könyvtárat.

INPUT:
- destination (3): a cél cím a 8MB -os lassú RAM-on belül ($000000 - $800000)
- source (3): a forrás cím a 256k méretű gyors RAM-on belül ($000000 - $100000)
- length (3): a másolandó blokk mérete (<$100000)

retVal:
- 0: ha a másolás rendben megtörtént
- $81: ha vagy a lassú, vagy a gyors RAMból kilógunk a másolás közben

Copy_dir_to_psram ($0F)

A pendrive (USB drive) aktuális könyvtár tartalmát bemásolja a lassú RAM-ba (PSRAM-be) és azonnal át is állítja az aktív drive -ot 1-re (PSRAM). Ez után a fileműveletek cél drive-ja a lassú RAM lesz, az USB port használható egérkezelésre.

Amennyiben a show_progress paraméter nem 0, akkor másolás előtt a képernyő 2színű text üzemmódba kapcsol, letörli azt, majd a másolás alatt a felső sorban látható az éppen bemásolás alatti file neve és mérete.

Sikeres másolás után a REG_DRIVE_PSRAM_VALID regiszter nem nullát ad vissza olvasásra.

INPUT:
- path (n) : TVC string, az aktuális drive -on egy könyvtár neve. Ha üres stringet kap paraméterül, akkor az aktuális könyvtár tartalmát tölti be (az aktuális drive-ról)
- show_progress (1): jelenjen-e meg a képernyőn a bemásolt file -ok nevei vagy sem. 0: nem, egyéb: igen

OUTPUT:
- progress (1): másolás közben ez a byte frissül azzal a számmal, amennyi file-t sikerült felmásolni a psram drive-ra. Amikor a rutin elindul, akkor kerül 0 a paraméter lista 0.

pozíciójára. (Ez az egyetlen rutin, amiről a TVC tudhat, h mikor *kezdődött el* a végrehajtása s nem csak a végéről értesülhet)

retVal:
- 0: sikeres másolás
- $80: PSRAM nem érzékelhető, nincs lassú RAM a rendszerben
- $81: nincs pendrive csatlakoztatva
- $8x-$9x: bármilyen IO hiba esetén a fatfs lib hibakódja + $80

Replace_pixel_color ($10)

Gyors RAM-ban a színt cserél (felső- és alsó nibble-t külön-külön) 16c bitmap képek esetén (bitmap gfx, 16c sprite vagy 16c font képek esetén).

INPUT:
- start_addr (3) : kezdőcím a gyors RAM-ban ($000000 - $0FFFFF)
- length (3) : a blokk mérete, ahol a színcserét végre kell hajtani
- color_from (1) : a lecserélendő szín ($00-$0F)
- color_to (1) : amire le kell a színt cserélni ($00-$0F)

Memory_move_chunks_from_block ($11)

Adott egy memória blokk, melyben a másolandó adat folyamatosan, hézagok nélkül helyezkedik el. A rutin ebből a blokkból darabokat (memória töredékeket) másol egy kijelölt címtől kezdődően, de a cím minden egyes töredék másolása után egy adott növekménnyel nő.

Például ezzel a fgv-el egy hívással megoldható, hogy scrollozás során egy bitmap image szélső 8 pixel-oszlopa bemásolódjon a képernyő - mondjuk - jobb szélére: 4 byte -ot (8 pixelt) bemásolva minden sor végére (128 byte-onként) egy 240x4 byte nagyságú területről.

INPUT:
- destination (3): a cím a gyors RAM-ban ahová a másolás elkezdődik
- source (3): a cím a gyors vagy a lassú RAM-ban, ahonnan a másolás elkezdődik. Ha a legmagasabb helyiértékű bit 1-re van állítva, akkor a rutin a PSRAM-ból másol
- chunk_size (2): a memória töredék mérete, amit másolunk. Ennyivel nő az egyes töredékek másolása között a forrás cím is.
- count (2): a memória töredékek száma, amennyit másolni kell
- increment (2): egyes memória töredékek másolása után mennyivel kell növelni a cél címet. (Ha pl bitmap image -be soronként másolunk, akkor 128 -at kell megadni paraméterül.)

retVal:
- 0: a másolás rendben megtörtént

- $81: töredék méret, száma vagy a növekmény paraméter értéke 0
- $82: ha a másolás célja kilóg a gyors RAM-ból
- $83: ha a másolás közben kilógunk a forrás címtartományból

Memory_move_chunks ($12)

Ez a rutin memória töredékeket másol egyik helyről a másikra, viszont itt a forrás- és a cél cím is ugyanannyival, a megadott cím-növekménnyel nő a töredékek másolása között. Ezt a fgv-t pl. akkor lehet használni, ha a memóriában egy, a képernyőn megjeleníthető képből jóval nagyobb van letárolva és abból másoljuk ki, a megfelelő pixelsort vagy oszlopot.

Amennyiben a forrás- és a célcím is a gyors RAM-ban van, akkor az átlapolódásokat a rutin figyelembe veszi és a töredékek másolásának a sorrendjét aszerint hajtja végre.

INPUT:
- destination (3): a cím a gyors RAM-ban ahová a másolás elkezdődik
- source (3): a cím a gyors vagy a lassú RAM-ban, ahonnan a másolás elkezdődik. Ha a legmagasabb helyiértékű bit 1-re van állítva, akkor a rutin a PSRAM-ból másol
- chunk_size (2): a memória töredék mérete, amit másolunk.
- count (2): a memória töredékek száma, amennyiszer másolni kell
- increment (2): egyes memória töredékek másolása után mennyivel kell növelni a forrás- és a célcímet.

retVal:
- 0: a másolás rendben megtörtént
- $81: töredék mérete, száma vagy a növekmény paraméter értéke 0
- $82: ha a másolás célja kilóg a gyors RAM-ból
- $83: ha a másolás közben kilógunk a forrás címtartományból

Mirror_sprite_phase ($13)

Egy vagy több sprite alakját tükrözi a memóriában, helyben. A sprite alakok nem közvetlenül következnek egymás után, hanem 2c esetben 64 byte-onként, 16c esetben 256 byte-onként - ahogyan a video render engine elvárja.

INPUT:
- phase_address(3): a sprite alak címe a gyors RAM-ban
- phase_count(1): mennyi sprite -ot kell tükrözni (egymás után a memóriában, a kezdőcímtől kezdve)
- color_mode(1): 2c (0) vagy 16c (nem 0) sprite-ot kell tükrözni
- flip_mode(1): vízszintesen (0) vagy függőlegesen (nem 0) kell tükrözni

retVal:
- 0: ha a phase_count 0, vagy a flip sikeres

- $81: ha a megadott paraméterek alapján az utolsó sprite kibővített fázisa (64 byte vagy 256 byte) kilógna a 256kB gyors RAMból

zx7Decompress ($14)

Egy zx7-tömörített blokkot csomagol ki egy megjelölt gyors, vagy lassú RAM területről egy gyors vagy lassú RAM területre, közvetlenül.

Túlcsordulásokat nem vizsgál a kártya, ez egy potenciálisan veszélyes művelet.

INPUT:
- destination (3): ahová a kicsomagolás történik. 3 byte -os érték, ha a legmagasabb bit 1, akkor a PSRAM-ba (8MB tartomány), egyébként a 256kB -os fastRAM-ba csomagol ki a rutin
- source (3): forrás cím, ahonnan a kicsomagolás kezdődik. Ugyanúgy, mint a destination esetben, a legmagasabb helyiértékű bit a PSRAM -ot jelöli ki forrás tartományként, egyébként pedig a 256kB -os fastRAM-ból csomagol ki a kártya

OUTPUT:
- decompressed size (4): a kicsomagolt adat mérete. Ez a méret a bemenő paraméterek *után* lesz megtalálható a memóriában, azaz nem írja felül a korábbi bemenő paramétereket, viszont 4 byte hosszú, azaz a fgv híváshoz érdemes legalább 10 byte-al számolni

retVal:
Minden esetben 0.

Grafikai függvények

A grafikai függvények számára a látható képernyő 0,0 pontja a képernyő bal felső sarka, és a 255,239 pedig a képernyő jobb alsó sarka. Ha egy koordináta 16 bites előjeles egész szám, akkor a negatív koordináták a 0;0 ponttól balra és/vagy följebb lesznek, egy virtuális képernyőn. A nem a képernyőre eső pontok, alakzatok természetesen nem jelennek meg a képernyőn.

A színeket minden esetben `irgb` formátumban kell megadni és ugyanígy adja vissza a kártya is.

A grafikai rutinok célbuffere (‘célképe’) mindig a fastRAMban található a `reg_function_bitmap_base` regiszter által kijelölt címen ($8000-al osztható kezdőcímen).

Get_pen_color ($15)

Visszaadja a rajzolásokhoz használt tinta színét.

OUTPUT:
- pen_color(1): a tinta aktuális színe

Ez a fgv mindig 0-val tér vissza.

Set_pen_color ($16)

Beállítja a rajzolásokhoz használt tinta színét.

INPUT:
- pen_color(1): a tinta színe

Ez a fgv mindig 0-val tér vissza.

Get_dot_color ($17)

A `reg_function_bitmap_base` regiszter által kiválasztott baseAddress címmel kijelölt bitmapban visszaadja egy adott pont (x; y) színét.

INPUT:
- x (1): a pont x koordinátája
- y (1): a pont y koordinátája (y kisebb kell legyen, mint maxY)

OUTPUT:
- color(1): az (x;y) által kijelölt pont színe a `reg_function_bitmap_base` regiszter által kijelölt tartományban.

retVal:
- 0: ha a megadott koordináta rendben van
- 1: ha y nagyobb vagy egyenlő mint maxY regiszter által kijelölt érték

Set_dot_color ($18)

A `reg_function_bitmap_base` regiszter által kijelölt bitmapban beállítja egy adott pont (x; y) színét.

INPUT:
- x (1): a pont x koordinátája
- y (1): a pont y koordinátája (y kisebb kell legyen, mint maxY)

retVal:
- 0: ha a megadott koordináta rendben van
- 1: ha y nagyobb vagy egyenlő mint maxY regiszter által kijelölt érték

Draw_line ($19)

A `reg_function_bitmap_base` regiszter által kijelölt bitmapban vonalat húz két kijelölt pont között a beállított tinta színnel. A pontok lehetnek a képernyőn kívül is.

INPUT:
- x1 (2): a kezdőpont x koordinátája. 16 bites, előjeles egész
- y1 (2): a kezdőpont y koordinátája. 16 bites, előjeles egész
- x2 (2): a végpont x koordinátája. 16 bites, előjeles egész
- y2 (2): a végpont y koordinátája. 16 bites, előjeles egész

A visszatérési érték mindig 0. Ha nagyon ügyesen nyomjuk a stopperóránkat, akkor észrevehetjük, hogy a vízszintes vonalakat (különösen a hosszabbakat) a rutin kicsivel gyorsabban rajzolja, mint a 'ferdéket'.

Draw_rectangle ($1A)

A `reg_function_bitmap_base` regiszter által kijelölt bitmapban téglalapot rajzol a megadott bal fölső csúcstól a megadott szélességgel és magassággal, az aktuális tinta színnel.

INPUT:
- x (2): a téglalap bal felső csúcsának az x koordinátája. 16 bites, előjeles egész
- y (2): a téglalap bal felső csúcsának az y koordinátája.. 16 bites, előjeles egész
- w (1): a téglalap szélessége, 8 bites, pozitív egész
- h (1): a téglalap magassága, 8 bites, pozitív egész

Fill_rectangle ($1B)

Ugyanúgy, mint a `draw_rectangle` rutin téglalapot rajzol, de itt az aktuális tinta színnel ki is tölti a téglalap belsejét.

INPUT:
- x (2): a téglalap bal felső csúcsának az x koordinátája. 16 bites, előjeles egész
- y (2): a téglalap bal felső csúcsának az y koordinátája.. 16 bites, előjeles egész
- w (1): a téglalap szélessége, 8 bites, pozitív egész
- h (1): a téglalap magassága, 8 bites, pozitív egész

Draw_multi_line ($1C)

A `reg_function_bitmap_base` regiszter által kijelölt bitmapban több, egymás utáni vonalat (törtvonal) rajzol a megadott pontokkal mint kezdő- és végpontokkal és az aktuális

tinta színnel. Ha az első és az utolsó pont megegyezik, akkor egy zárt alakzatot, poligont kapunk.

INPUT:
- N (1): a pontok száma. Legalább 2

Ez után az alábbi blokk n-szer ismétlődik:

- x_n(2): az n. vonal kezdőpontjának x koordinátája (egyben az (n-1). vonal végpontjának az x koordinátája - amennyiben n>1). 16 bites, előjeles egész szám.
- y_n(2): az n. vonal kezdőpontjának y koordinátája (egyben az (n-1). vonal végpontjának az y koordinátája - amennyiben n>1). 16 bites, előjeles egész szám.

Látható, hogy a paraméter lista N*4 + 1 byte hosszúságú lesz, ahol N legalább 2 (azaz legalább 9 byte).

A visszatérési érték mindig 0.

Scanline_flood_fill ($1D)

A `reg_function_bitmap_base` regiszter által kijelölt bitmapban kifesti (kiönti) az aktuális tintaszínnel azt a területet a megadott ponttal kezdődően, ahol a pontszín megegyezik a megadott pont színével. Nyakatekert, de ez az egyszerű flood fill rutin. Zárt alakzatoknak lehet így a belső színét módosítani.

A fejlesztés során úgy találtam, hogy ez az implementáció memória takarékosabb megoldás, mint az egyszerű, stack-es kitöltés.

INPUT:
- startX(1): a fill kezdőpontjának x koordinátája (8 bites, pozitív egész szám)
- startY(1): a fill kezdőpontjának y koordinátája (8 bites, pozitív egész szám). Nem lehet nagyobb, mint a képernyő magassága

retVal:
- 0: a kitöltés rendben megtörtént
- 1: a megadott y koordináta kilóg a képernyőről
- 2: memória túlcsordulás történt (volna), a flood fill valahol félúton abbamaradt, túl nagy memória igény miatt.

Draw_ellipse ($1E)

A `reg_function_bitmap_base` regiszter által kijelölt bitmapban ellipszis rajzolása a megadott középpontban a (szélesség/2) és b (magasság/2) paraméterekkel. Az ellipszis kilóghat a képernyőről.

INPUT:
- center_x (2): A középpont x koordinátája, 16 bites, előjeles egész szám
- center_y (2): A középpont y koordinátája, 16 bites, előjeles egész szám
- a (1): az ellipszis szélességének a fele, 8 bites, pozitív egész szám
- b (1): az ellipszis magasságának a fele, 8 bites, pozitív egész szám

Mindig 0-val tér vissza.

Fill_ellipse ($1F)

A draw_ellipse -hoz hasonlóan ellipszist rajzol, viszont ki is tölti azt az aktuális tinta színnel.

Copy_image_block ($20)

A `reg_function_bitmap_base` regiszter által kijelölt bitmapre képet másol úgy, hogy a forráskép transzparens pixelei nem módosítják a célképet (ott átlátszó lesz a kép).

A forráskép sorai egymás után, sorfolytonosan helyezkedik el a memóriában. A képnek egy sora mindig egész byte-ra van kiegészítve (páratlan pixel szélességű kép sorai végén mindig lesz egy 'felesleges' pixel - de azt nem másolja be a rutin.

Nem használható, ha a képet átfedéssel önmagára másoljuk.

INPUT:
- destX(2): a cél x koordináta. 16 bites, előjeles egész
- destY(2): a cél y koordináta. 16 bites, előjeles egész
- width(1): a forráskép szélessége, 8 bites, pozitív egész (max 255)
- height(1): a forráskép magassága, 8 bites, pozitív egész (max 255)
- sourceAddr(3): a forrás címe a memóriában, ahonnan másolni kell a kép adatait. Ha a legfelső bit 1, akkor PSRAM, egyébként fastRAMból másol. Ha fastRAM a forrás, akkor az érték nem szabad nagyobb lennie, mint 256kB

A visszatérési érték mindig 0.

Copy_sub_image ($21)

Egy képből másol ki egy kisebb darabot. Transzparens pixelek kezelése opcionális. Ez a másolás hasonlít a `memory_move_chunks` fgv -hez több különbséggel:
- a cél és a forrás területen ott ugyanannyi a növekmény a másolás során, itt nem, a cél és a forrás képnek lehet más a mérete
- az a fgv teljes byte-határról másol, itt pedig a kezdő pixelt is megadhatjuk
- itt mind a cél-, mind a forrás pontos kezdőcímét ez a fgv számolja ki a képernyőkoordináták alapján.

Nem használható, ha a képet átfedéssel önmagára másoljuk.

INPUT:
- destX(2): a cél x koordináta. 16 bites, előjeles egész
- destY(2): a cél y koordináta. 16 bites, előjeles egész
- dWidth(1): a kép szélessége, amibe másolunk, 8 bites, pozitív egész (max 255)
- dHeight(1): a kép magassága, amibe másolunk, 8 bites, pozitív egész (max 255)
- srcX(2): a forrásképből az x koordináta, ahonnan másolunk (bal felső sarka a másolandó képdarabnak). 16 bites, előjeles egész
- srcY(2): a forrásképből az y koordináta, ahonnan másolunk (bal felső sarka a másolandó képdarabnak). 16 bites, előjeles egész
- sWidth(2): a kép szélessége, ahonnan másolunk. 16 bites, pozitív egész
- sHeight(2): a kép magassága, ahonnan másolunk. 16 bites, pozitív egész
- sourceAddr(3): a forráskép kezdőcíme a memóriában, ahonnan másolni kell a kép adatait. Ha a legfelső bit 1, akkor PSRAM, egyébként fastRAMból másol. Ha fastRAM a forrás, akkor az érték nem szabad nagyobb lennie, mint 256kB
- opts_defColor(1): alsó 4 bit kijelöli azt a színt, amit azok a pixelek kapnak, amik nincsenek a forrásképben. Ha a felső 4 bit nulla, akkor minden pixel megjelenik a célképen, ahogyan az a forrásképben van. Ha a felső 4 bit nem nulla, akkor a transparans pixelek nem változtatnak a célképen.

PSRAM drive műveletek

Históríkus okokból a copy_dir_to_psram nem itt szerepel, de logikailag az is ide értendő..

Create_psram_drive ($22)

Létrehoz egy üres PSRAM drive -ot a paraméterül megadott mérettel a slowRAM tartomány elejétől kezdve. Egy esetlegesen már létező PSRAM drive-ot letörli. Ha megfelelően méreteztük a drive -t, akkor innentől lesz egy használható memória (PSRAM) drive-unk. Ha a drive nem foglalja el a teljes PSRAM területet, akkor a mögötte lévő, fennmaradó memória terület használható marad a felhasználó számára. Sikeres végrehajtás után a REG_DRIVE_PSRAM_VALID regiszter nem nullát ad vissza olvasásra.

INPUT:
- size_in_blocks: a létrehozandó slow RAM (PSRAM) drive mérete, 4kB -os (4096 byte -os) blokkokban. Ez 8MB méretű PSRAM esetén legfeljebb 2048 block lehet. Ha ennél nagyobb méretet adunk, akkor a fgv hibával fog visszatérni.

OUTPUT: nincs

retVal:
- 0: ha a drive-t sikerült létrehozni

- 255: ha túl nagy méretet adtunk meg (-1)
- err_code: valamilyen LFS hibakód

Get_first_usable_psram_pos ($23)

Visszaadja azt a pozíciót a slow RAM-ban, ahonnan kezdve biztonságosan használhatjuk azt, nem megsértve a korábban létrehozott psram drive -ot.

INPUT: nincs

OUTPUT:
- first_usable_psram_pos (3): a psram tartomány azon pozíciója, amibe már nem nyúlik bele egy esetlegesen létrehozott psram drive. Ez az érték 0, ha nincs psram drive létrehozva, esetleg 0x00FFFFFF, ha már nincs szabad hely a létező psram drive mellett (mögött/után).

retVal: mindíg 0

Delete_psram_drive ($24)

Ha létezett korábban PSRAM drive, akkor annak tartalmát megsemmisíti és felszabadítja (FF -el tölti fel a korábban lefoglalt drive területét és az első szabad pozíció a PSRAM-on 0 lesz). A fgv végrehajtása után REG_DRIVE_PSRAM_VALID regiszter nullát ad vissza olvasásra.

INPUT: nincs

OUTPUT: nincs

retVal: mindíg 0

Egyszerű matematikai kifejezés kiértékelése

Lehetőség van egy nem túl bonyolult matematikai kifejezés kiszámítására. A belső implementáció 32 bites FLOAT -ot használ, ami kevésbé pontos, mint a TVC belső FLOAT számábrázolása, viszont annál gyorsabban számol.

Evaluate_math_expression ($25)

Fordított lengyel jelöléssel beadott matematikai kifejezések kiértékelése. A bemenő paraméter lista egy számokból, előre kijelölt konstansokból és matematikai műveletkből álló sorozat, melyet egy 0 zár. A kimenet pedig valamilyen egész vagy FLOAT típusú változó.

Számok

Minden számot úgy adunk meg, hogy az első byte-on kijelöljük a szám típusát, majd magát a számot (egy vagy több byte-on). A következő számtípusokat ismeri fel a kártya:

- UINT8, kódja: 64 - egy byte-on ábrájolt, előjel nélküli egész szám
- INT16, kódja: 65 - két byte-on ábrázolt, előjeles egész szám

- INT32, kódja: 66 - négy byte-on ábrázolt, előjeles egész szám
- TVC-FLOAT, kódja: 67 - 6 byte -on ábrázolt, TVC natív lebegőpontos szám. Formátuma megtalálható a tvc64k_plus.pdf könyv 29-es (16.) oldalán, a szimbólumtábla alatti leírásban. BASIC programból egy változó memóriában tárolt címe a VARPTR() fgv segítségével határozható meg. Ez a típus csak bemenő paramétere lehet egy fgv kiértékelésnek
- FLOAT, kódja: 68 - 4 byte-on ábrázolt float szám, ahogyan az MCU tárolja a 32 bites lebegőpontos számokat
- VARREF, kódja 69 - 3 byte-on ábrázolt pointer a fastram-ban egy számra (számtípussal együtt, konstansra sem hivatkozhatunk). Ezzel a típussal lehetőségünk van arra, hogy egy kifejezésben többször szereplő számot, vagy egy másik kiértékelés eredményét, stb... meghivatkozzuk, így nem kell direktben bemásolni a kifejezésbe a számot, amit szeretnénk használni.

Konstansok

Vannak előre definiált FLOAT konstansok, melyek minden esetben 1 byte -osak:

- 2π, kódja: 128
- π, kódja: 129
- $\pi/2$, kódja: 130
- 0.001, kódja: 131
- 0.01, kódja: 132
- 0.1, kódja: 133
- 1.0, kódja: 134
- 10.0, kódja: 135
- 100.0, kódja: 136
- 1000.0, kódja: 137

Műveletek

A kiértékelő rutin által felismert matematikai operátorok (függvények) kódjai minden esetben 1 byte-osak:

- FINISH (0) - a kifejezés lezárása
- ADD (1) - két szám összege. Ha bármely paramétere FLOAT vagy TVC-FLOAT, akkor az eredmény FLOAT lesz. Máskülönben az eredmény a paraméterek közül a több byte-ból álló egész lesz. Pl: 16 1 16 2 1 0 (1+2 => UINT8: 1, UINT8: 2, +)
- SUB (2) - két szám különbsége. Az eredmény típusa, mint az összeadásnál Pl: 16 2 16 1 2 0 (2-1 => UINT8 2, UINT8 1, -)

- MUL (3) - két szám szorzata. Az eredmény típusa, mint az összeadásnál. Pl: 16 2 17 0 1 3 0 (2*256 => UINT8 2, INT16 256, *)
- DIV (4) - két szám hányadosa. Az eredmény típusa, mint az összeadásnál. Pl: 17 0 1 16 2 3 0 (256/2 => INT16 256, UINT8 2, /)
- MAX (5), MIN(6) - két szám közül kiválasztja nagyobbat/kisebbet. Az eredmény típusa, mint az összeadásnál
- ABS (7) - a paraméterének az abszolút értékét adja vissza, a bemenő paraméter típusában
- SQR (8) - a paraméterének a négyzetét számolja ki. Az eredmény típusa megegyezik a paraméter típusával.
- SQRT (9) - a paraméterének a négyzetgyökét számolja ki. A kimenő paraméter minden esetben FLOAT lesz
- SIN (10), COS(11), TAN(12), szögfüggvények, a paraméterüket radiánban várják. Ha a paraméter nem FLOAT vagy TVC-FLOAT, akkor hibával kilépnek. A visszatérési érték típusa minden esetben FLOAT lesz.
- TOINT (13) - a paraméterének veszi az egész részét és INT32 típusra konvertálja. Hibát ad, ha nem FLOAT vagy TVC-FLOAT típusú paramétert kapott.
- TOFLOAT (14) - a paraméterül kapott egész számot FLOAT -ra konvertálja. Hibát ad, ha nem egész típusú számot kapott paraméterül
- ROUND (15) - a paraméterül kapott FLOAT vagy TVC-FLOAT típusú számot kerekíti a hozzá legközelebb eső egész számra. .5 értéket a nullától messzebb eső egész számra kerekíti.

Kiértékelés végrehajtása

A kiértékelő rutin a kiszámítandó kifejezést fordított lengyel jelölésben várja. Ezt úgy kell elképzelni, hogy az input paramétereken megy végig a rutin: ha számot (vagy konstanst) talál, akkor azt lerakja a vermébe, ha műveletet talál, akkor a verem tetejéről kiveszi a művelet paramétereit (számokat, konstansokat), elvégzi a műveletet, majd az eredményt visszarakja a verembe. A kifejezés visszatérési értéke a verem tetején található szám lesz. Ha helyes a kifejezésünk, akkor a kiértékelés végére egyetlen szám lesz a veremben: a végeredmény.

A kiértékelés során használt stack maximális mélysége 32. Ha ezt meghaladja egy kifejezés, akkor a kiértékelés hibával ér véget.

Ha multi_execute hívásban szerepel ez a fgv hívás, akkor figyeljünk rá, hogy legalább 5 byte -ot hagyjunk ki a következő fgv híváshoz, mert a kiértékelő rutin előre nem tudja, h milyen típusú lesz a visszatérési érték - 5 byte -ot feltételez mindenképpen (<típus> + <4db érték byte>). Ha 5 byte-nál nagyobb a paraméterlista, akkor természetesen nem kell több helyet hagyni.

Lássunk néhány példát:

a) 10 + 20

Fordított lengyel jelöléssel a így néz ki: 10 20 +

A helyes paraméterlista így néz ki: 64 (szám típusa: uint8) 10 (érték) 64 (uint8) 20 (érték) 1 (ADD) 0 (a kifejezés vége). Azaz 64 10 64 20 1 0

b) (10 + 20) * (400 - 30)

Egyrészt itt már figyelnünk kell a zárójelekre, másrészt a 400 nem fér bele az UINT8 típusba. Mindenesetre a kifejezés így néz ki FLJ -sel: `20 40 + 400 30 - *`

Mi történik? 20 és 40 stack -re kerül, eljut a gép a + operátorhoz. Kiveszi a stackből a két számot, összeadja, visszarakja a stack -be a 60-at. Aztán lerakja stack-re a 400-at és a 30 -at (a stackben: 30 400 60 van). A kiértékelő eljut a - operátorig, ezért kiveszi a két felső elemet végrehajtja a műveletet (figyelem, fontos a sorrend!) és visszarakja a 400-30 eredményét (370). A stack -en a két zárójeles kifejezés eredménye van. A következő elem a sorban a * operátor, ezért kiveszi a két elemet, összeszorozza, majd az eredményt visszarakja a stack -be. A helyes paraméterlista így néz ki: `64 20 64 40 1 65 144 1 64 30 2 3 0`

c) 5 * 0.001 * 2π

Itt érdemes az előre definiált konstansokat használni: 0.001: 131, 2π: 128

`64 5 131 3 128 3 0`

Ha a kapott eredményt (0.031415... => `68 253 173 0 61`) bemásoljuk egy másik kifejezésbe, akkor azt felhasználhatjuk egy SIN fgv kiértékelésénél, amennyiben még azt megszorozzuk egy 0-199 közötti futó értékkel:
ROUND(SIN(**5 * 0.001 * 2π** * ***i***) * 100 + 100)

`68 253 173 0 61 64 i` `3 9 136 3 136 1 15 0`

(Itt a `136` az egy konstanst jelöl: 100)

A fenti számsorba ***i*** helyére a 0-199 közötti számunkat írjuk, a végeremény egy 0 és 200 közötti (inklúzív) egész szám lesz.

Az első 5 szám a fenti szorzatunk eredménye. Utána megadjuk ***i*** változónkat (UINT8 formában), összeszoroztatjuk őket (3) majd kiszámoljuk a szorzat szinuszát (9). Az eredményt megszorozzuk 100.0-val (136 3) majd hozzáadunk százat (136.0 1) végül az eredményünket kerekítjük egészre.

INPUT:
- math_expression (n): nullával zárt matematikai kifejezés az előbbiekben leírt formában

OUTPUT:
- <type> <value> : egy szám, aminek a típusa UINT8, INT16, INT32 vagy FLOAT lehet. A type ennek megfelelően van beállítva. A <value> 1, 2 vagy 4 db byte, ami kifejezi a beadott

kifejezés értékét. UINT8 típus esetén 1 byte, INT16 esetén 2 byte, INT32 és FLOAT esetben 4 byte-on van leírva a végeredmény.

Grafikai függvények II - két színnel

Az alábbiakban olyan grafikai függvények vannak, amik a zárt alakzatukat két színnel töltik ki, pepita elrendezésben.

Fill_triangle_2c ($26)

Ez a függvény egy 3 szöget rajzol a képernyőre és kitölti azt pepitában a két megadott színnel. A háromszöget csak abban az esetben rajzolja meg, ha a csúcsai az óramutató járásával ellentétes irányban vannak definiálva (azaz egy 3D forgatásos transzformáció után, ha a 3szög hátrafelé néz, akkor nem fog kirajzolódni).

INPUT:
- x0, y0, x1, y1, x2, y2 (6): 3 csúcspont x és y koordinátája, előjeles, 16 bites formában. Ha a csúcsok óramutató járásával egyező irányban lettek meghatározva, akkor a háromszög nem rajzolódik ki.
- two_colors (1): 2db 4 bites szín, a paraméter byte alsó és a felső 4 bitjén meghatározva. Minden páros sorban a páros y koordínátájú pontok színe a felső 4 bit, a páratlan y-ra pedig az alsó 4 bit színű pont jelenik meg. Páratlan sorokban ez fordítva van.

Amennyiben a pen_color nem 8 (transzparens) színre van állítva, akkor a háromszög oldalai is kirajzolódnak a besatírozott rész körül a pen_color által meghatározott színnel (de csak akkor, ha a csúcspontok az óramutató járásával ellentétes irányban vannak).

Fill_box_2c ($27)

Ez a függvény ugyanúgy működik, mint az egyszínű fill box függvény, csak van egy extra two_colors paramétere, ami a kiszínezést meghatározza. A csúcspontok sorrendjére nincs semmilyen megkötés.

Amennyiben a pen_color nem 8 (transzparens) színre van állítva, akkor a téglalap oldalai is kirajzolódnak a besatírozott rész körül a pen_color által meghatározott színnel.

INPUT:
- x, y (4): a téglalap bal felső csúcsának a koordinátái, 16 bites, előjeles egészként
- w, h (2): a téglalap szélessége és magassága 8 bites előjel nélküli számokkal

Érdekesség, hogy a szélessége és a magassága a téglalapnak 1-el nagyobb értéket jelöl. Azaz ha például 0 szélességet határozunk meg egy téglalapnak, akkor csak 1 függőleges szakaszt kapunk - ami 1 pixel széles. Ha a szélességre 1 -et adunk meg, akkor egy olyan függőleges vonalat kapunk, aminek a szélessége 2 pixel, stb...

Scanline_flood_fill_2c ($28)

Ez a függvény ugyanúgy működik, mint az egyszínű párja, csak nem egy színnel, hanem kettő színnel, pepitában színezi ki az adott területet.

Arra oda kell figyelni, hogy a rutin nem tolerálja, ha olyan színű tartományt kell átszínezni, amelynek a színe megegyezik a megadott 2 szín egyik színével. Kisebb területet még ki tud színezni, de hamar zátonyra fut, mivel könnyen megtelhet a belső stack.

INPUT:
- x, y (2): a színezés kiindulópontja előjel nélküli 8 bites számokkal megadva. Y nem lehet nagyobb a beállított terület magasságától
- two_colors(1): két szín, a paraméter alsó illetve felső 4 bitjén megadva

retVal: ugyanaz, mint az egyszínű scanline_flood_fill rutinnál.

Memória blokkok másolásának kibővítése

Memory_move_chunks_2increments ($29)

Ez a rutin a memory_move_chunks($12) általánosított formája: itt a source és a destination növekményét meg lehet adni, azaz külön-külön megadható, hogy a src/dst mennyivel növekedjen 1-1 blokk (chunk) másolása után.

INPUT:
- destination (3): a cím a gyors RAM-ban ahová a másolás elkezdődik
- source (3): a cím a gyors vagy a lassú RAM-ban, ahonnan a másolás elkezdődik. Ha a legmagasabb helyiértékű bit 1-re van állítva, akkor a rutin a PSRAM-ból másol
- chunk_size (2): a memória töredék mérete, amit másolunk.
- count (2): a memória töredékek száma, amennyiszer másolni kell
- incrementDest (2): egyes memória töredékek másolása után mennyivel kell növelni a célcímet
- incrementSrc (2): egyes memória töredékek másolása után mennyivel kell növelni a forrás címet

retVal:
- 0: a másolás rendben megtörtént
- $81: töredék mérete, száma vagy a növekmény paraméter értéke 0
- $82: ha a másolás célja kilóg a gyors RAM-ból
- $83: ha a másolás közben kilógunk a forrás címtartományból

File műveletek

A fileműveletek mindig a REG_DRIVE_SELECT regiszter által kijelölt drive-ra értendőek. Az USB drive (0), a lassú RAM-ban található PSRAM drive (1) és a DSK drive* (3) írható és olvasható míg a flash drive (2) csak olvasható drive, azt nem tudja írni (változtatni) a kártya. A módosításra irányuló fgv -ek hibával fognak visszatérni.

*) a DSK drive csak akkor írható, ha a forrás file vagy USB vagy PSRAM drive -on van.

Óriási trükk, hogy az open_file és az open_dir által visszaadott 4 byte-os file és dir handle a kimeneti paraméterek elején helyezkedik el ott, ahol a következő file vagy dir-művelet a bemenő paramétereit várja (és mindig elsőként várják a file vagy dir handle-t). Ha csak 1 file-al dolgozunk, akkor ezt a 4 byte -os azonosítót nem kell mozgatni, másolgatni.

Az alábbi file-műveletek végezhetőek el a REG_FUNCTION_SELECT regiszter segítségével:

Open_file ($81)

File megnyitása olvasásra vagy írásra. Írásra csak az USB, PSRAM és DSK drive -on lehet file -t megnyitni / adatot menteni.

INPUT:
- mode (1): file mode
- type (1): ha .cas file-t kell létre hozni, a cas file típusa: $01 vagy $11 (buffered, un-buffered)
- filename (n+1): a file neve tvc string formában. String hossza + karakterek.

A mode paramétere a következő értékek binárisan VAGY-olt összege (forrás: fatFS library, ff.h, FA_nnn)

- $00: OPEN_EXISTING - létező file megnyitása
- $01: READ - olvasásra nyitás
- $02: WRITE - írásra nyitás
- $04: CREATE_NEW - új file létrehozása
- $08: CREATE_ALWAYS - file létrehozása, ha létezett, akkor a tartalmát törli
- $10: OPEN_ALWAYS - a file létrehozása, ha létezett, akkor elkezdi felülírni a tartalmát
- $30: OPEN_APPEND - a létező file-t megnyitja és a file-hoz hozzáír (auto seek a végére)

Példák:

- Létező, olvasásra nyitandó file: $01
- Létező, felülírandó file: $0a
- Új file-t szeretnénk létrehozni, ha már létezik, legyen hiba: $06

Olvasásra nyitás esetén:

- A file neve végződhet * -al, ekkor az első olyan file-t fogja megnyitni a kártya, ami a * előtti stringgel egyezik.

- A file neve nem feltétlenül kell tartalmazzon kiterjesztést. Ekkor az első illeszkedő `.cas` file-t fogja megnyitni a kártya. Vegyük észre, h a `filenév*` esetben nem csak a `.cas` file-ok között keres a kártya (hanem pl. megtalálja a `filenév.bin` file -t is, amit a LOAD parancs nem fog tudni betölteni)
- A `type` paramétert figyelmen kívül hagyjuk.
- Ha a megtalált file `.cas` file, akkor az első 128 byte -ot a kártya a megnyitás során csendben átugorja.

OUTPUT:
- file_handle (4): egy 4 byte hosszú file azonosító. Ezzel a 4 byte -al lehet egy korábban már megnyitott file-on műveleteket végezni. Egy drive -on egyszerre 4 file lehet nyitva.
- file_size (4): a megnyitott file mérete. Írás esetben ez a mező nincs kitöltve. Amennyiben a file nagyobb, mint 2^32-1 (exFat usb drive esetben), akkor a méret fixen 2^32-1 értéket fog tartalmazni. Flash drive esetben a legnagyobb file méret 2^30-1.
- file_name: a megtalált file valódi neve, tvc string formában (max 256 byte)

retVal:
- 0: file open sikeres
- hibakód + $80: hiba történt

Close_file ($82)

Egy korábban megnyitott file lezárása.

INPUT:
- file_handle(4) : a korábban megnyitott file azonosítója

retVal:
- 0: ha a file lezárása sikeres
- hibakód + $80: bármilyen hiba esetén

Read_file ($83)

Egy korábban (olvasásra) megnyitott file -ból olvas egy adatblokkot. A blokkméret maximum kb 1.5kB (6x256 - 2), de arra mindenképpen figyelni kell, hogy a fgv a beolvasott blokkot a kijelölt paraméter területre fogja másolni.

INPUT:
- file_handle(4): a megnyitott file azonosítója
- block_size (2): a beolvasandó blokk mérete. Maximum 1534 byte
- buffer (n): ide fogja olvasni a file tartalmát a kártya

OUTPUT:
- file_handle(4) : a megnyitott file azonosítója, a fgv érintetlenül hagyja

- read_bytes(2) : a beolvasott byte -ok száma
- buffer (read_bytres): a beolvasott adat

retVal:
- 0: ha az olvasás hiba nélkül lezajlott
- hibakód + $80

Write_file ($84)

Egy korábban (írásra) megnyitott file-ba ír egy adatblokkot. A blokkméret maximum kb 1.5kB lehet (6x256-2). Csak USB, PSRAM és DSK drive -on lehet file -t írni.

INPUT:
- file_handle(4): a megnyitott file azonosítója
- block_size(2): a kiírandó blokk mérete. Maximum 1534 byte
- block (block_size): a kiírandó byte-ok

OUTPUT:
- file_handle(4): a megnyitott file azonosítója, a fgv érintetlenül hagyja
- written_bytes(2): a kiírt byte-ok száma. Jó esetben ugyanannyi, mint amit az INPUTnál megadott a hívó

retVal:
- 0: ha az írás hibátlanul megtörtént
- $8a: csak olvasható drive
- fatFS hibakód + $80

Open_dir ($85)

Könyvtár (directory) megnyitása olvasásra. Mielőtt egy könyvtár tartalmát beolvasnánk bejegyzésenként, egyesével, előtte a könyvtárat meg kell nyitni, majd munkánk végeztével le kell azt zárni - ahogyan egy file-t is - a close_dir függvénnyel.

INPUT:
- path(n+1): egy tvc string, ami a megnyitandó könyvtár útvonalát tartalmazza. Ha nem / jellel kezdődik, akkor a könvytár relatív útvonalként lesz értelmezve. A relatív útvonalak kezelését a slowRAM és flash drive esetben a kártya implementálja, USB drive esetben pedig a fatFS library. Különbségek lehetnek, az egyszerű esetek viszont azonosan működnek.

OUTPUT:
- dir_handle(4): a megnyitott directory azonosítója

retVal:
- 0: ha a könyvtárat sikerült megnyitni
- hibakód + $80: egyéb esetben

Close_dir ($86)

Egy korábban megnyitott könyvtár lezárása.

INPUT:
- dir_handle(4): a megnyitott könyvtár azonosítója

retVal:
- 0: a könyvtárat sikeresen lezárta a kártya
- hibakód + $80: egyébként

Read_dir ($87)

Egy könyvtárbejegyzés olvasása a megnyitott könyvtárból. Abban az esetben, ha elértük a már beolvastuk a könyvtár utolsó bejegyzését is, akkor a file_name 0 hosszúságú lesz (első byte 0).

INPUT:
- dir_handle(4): a megnyitott könyvtár azonosítója

OUTPUT:
- dir_handle(4): a megnyitott könyvtár azonosítója, a fgv érintetlenül hagyja
- file_name (256): tvc stringként a file vagy könyvtár neve, maximum 255 hosszú lehet egy név. Ez az output mező fixen 256 byte hosszú.
- file_type(1): a bejegyzés típusa, USB drive esetben FatFS library, ff.h szerint (01-RO, 02-HID, 04-SYS, 10-DIR, 20-ARC), slowRAM -ban csak $01 (file, readonly), flash drive: $01 (file, readonly) vagy $10 (directory) lehetséges.
- file_size(4): ha file típusú a bejegyzés, akkor a file mérete. Egyébként nincs értelmezve. Flash drive esetben max 2^30-1, USB és slowRAM drive esetén max 2^31-1.
- short_name(13): Csak akkor van kitöltve, ha a file_name hosszabb, mint 12 karakter (nem fér bele a 8.3 formátumba). Ezzel a névvel az USB drive engedi a fileműveleteket, a slowRAM és flash drive -ok esetén csak jelzés értékűek (nincs implementálva a ‘fileOpen on short name’ feature).

retVal:
- 0: ha a könyvtár bejegyzést sikerült beolvasni. Figyelem, 0 jön vissza akkor is, ha már vége a katalógusnak!
- hibakód + $80

Write_file_source ($88)

Adatblokk írása korábban megnyitott file-ba fast- vagy slowRAM területről. A blokk méretének felső korlátja fastRAM esetben: 256kB. Gyorsabb művelet, mint a sima write_file, mivel abban az esetben a kiírandó adatot át kell másolni egy, az MCU által is olvasható memória területre (az a terület pedig az IO ROM felső 1.5kB -ja). Csak USB, PSRAM és DSK drive -on lehet file -t írni.

INPUT:
- file_handle(4): az írásra megnyitott file azonosítója
- block_len(3): a kiírandó blokk mérete
- source(3): a kiírandó blokk kezdőcíme a fastRAM területen ($000000-$0003FFFF). Ha slowRAM -ot szeretnénk kijelölni, akkor a 23. bitet 1-re kell állítani s úgy beállítani a max 8MB -os területről a forrás címet (pl: 0x8A0000, ami a PSRAM 0x000000-0x7FFFFF tartományából a 0x0A0000 címét jelöli ki)

OUTPUT:
- file_handle(4): az írásra megnyitott file azonosítója - változatlan
- written_bytes(3): a sikeresen kiírt byte -ok száma

retVal:
- 0: az írás sikeres volt
- $8a: csak olvasható drive
- FatFS hibakód + $80: hiba történt írás közben

Read_file_dest ($89)

Adatblokk beolvasása a korábban olvasásra megnyitott file-ból a megadott fastRAM vagy slowRAM területre. FastRAM kijelölés esetén a blokk nem lóghat ki a fastRAM területről (max 256kB). Gyorsabb művelet, mint a read_file, mivel abban az esetben a beolvasott adatot be kell másolni a paraméter területre. Arról nem is beszélve, hogy ez a fgv nagyobb blokkokat is be tud olvasni. PSRAM célcím kijelölése esetén a programozó feladata, hogy ne írja felül egy esetlegesen létező PSRAM drive adatait.

INPUT:
- file_handle(4): az olvasásra megnyitott file azonosítója
- block_len(3): a beolvasandó blokk mérete (max 256kB)
- destination(3): a célcím a a fastRAM területen (0x000000-0x0003FFFF). Ha slowRAM -ot szeretnénk kijelölni, akkor a 23. bitet 1-re kell állítani s úgy beállítani a max 8MB -os területről a célcímet (pl: 0x8A0000, ami a PSRAM 0x000000-0x7FFFFF tartományából a 0x0A0000 címét jelöli ki). PSRAM

OUTPUT:
- file_handle(4): az olvasásra megnyitott file azonosítója - változatlan
- read_bytes(3): a beolvasott byte-ok száma

retVal:
- 0: ha az olvasás sikeres volt. Még akkor is 0, ha elértük a file végét és nem olvastunk vissza annyi byte -ot, mint ami a block_len -ben meg volt adva
- hibakód + $80: ha az olvasás nem volt sikeres

File_seek ($8a)

A megnyitott file-t a megadott pozícióba állítja. A következő olvasás (írás) ugyanerre a file-ra onnan fog történni.

INPUT:
- file_handle(4): a megnyitott file azonosítója
- file_pos(4): az új file pozíció (alacsony helyiértéktől a magas felé)

OUTPUT:
- file_handle(4): a megnyitott file azonosítója - változatlan

terVal:
- 0: ha a seek sikeres
- hibakód + $80: ha a seek nem volt sikeres

Getcwd ($8b)

Visszaadja az aktuális munkakönyvtárat.

Nincs input paraméter.

OUTPUT:
- current_working_dir (1+n): az aktuális munkakönyvtár, tvc string, max 255 byte hosszú

retVal:
- 0: ha sikeres volt a lekérdezés
- hibakód+$80: hiba esetén

Chdir ($8c)

Az aktuális munkakönyvtár váltása. PSRAM drive nem implementálja ezt a fgv -t.

INPUT:
- new_path: tvc string, az új könyvtár neve. Ha / jellel kezdődik, akkor abszolút könyvtárat jelölünk ki. Ha a jelenlegi könyvtáron belül akarunk új könyvtárba váltani, akkor engedélyezett a * jel a könyvtár név végén és akkor az első, illeszkedő könyvtárba fog beváltani a kártya az adott drive-on (USB vagy Flash drive).

retVal:
- 0: a könyvtár váltás sikeres
- hibakód + $80: hiba a könyvtárváltás során

Mkdir ($8d)

Új könyvtár létrehozása. Csak USB, PSRAM és DSK drive esetén implementált.

INPUT:
- dir_name (1+n): tvc string, a létrehozandó könyvtár neve. Lehet abszolút vagy relatív.

retVal:
- 0: a könyvtár létrehozása sikeres
- hibakód + $80: hiba a könyvtár létrehozása során

Delete ($8e)

File, vagy (üres) könyvtár törlése. Csak USB, PSRAM és DSK drive esetén implementált.

INPUT:
- file_name (1+n): tvc string, a törlendő file vagy könyvtár neve

retVal:
- 0: a törlés sikeres
- hibakód + $80: hiba a törlés során

Rename ($8f)

File vagy könyvtár átnevezése. Csak USB, PSRAM és DSK drive esetén implementált.

INPUT:
- name_from (1+n): tvc string, az átnevezendő file / könyvtár neve
- name_to (1+m): tvc string, az új neve a file-nak / könyvtárnak

retVal:
- 0: az átnevezés sikeres
- hibakód + $80: hiba az átnevezés során

Getstat ($90)

Egy file létezését, attribútumait lehet vele lekérdezni.

INPUT:
- file_name (1+n): tvc string, a lekérdezendő file neve

OUTPUT:
- file_name (1+n): tvc_string, a file teljes neve (USB, flash: max 255 hosszú, PSRAM: max 31 hosszú)
- file_attrib(1): lásd dir entry
- file_size(4): file mérete

- short_filename(13): amennyiben a file_name hosszabb, mint 13. Csak USB drive esetén használható fel OPEN hívásra.

retVal:
- 0: file/dir létezik, OUTPUT kitöltve
- hibakód+$80: file nem létezik, vagy valami hiba történt

Sync ($91)

A filerendszerre írt adatot az esetleges memória cache -ből valóban kiírja az USB drive-ra. Az írásra megnyitott file CLOSE hívása automatikusan kiírja az adatokat a drive -ra. Csak USB drive esetén implementált.

retVal:
- 0: a sync sikeres
- hibakód+$80: minden egyéb esetben

Mount DSK ($92)

Egy .dsk file -t csatol fel és abból létrehozza a DSK Drive -ot. Sikeres művelet után a REG_DRIVE_DSK_VALID regiszter nem nullát ad vissza olvasásra.

INPUT:
- file_name (1+n): tvc string, a .dsk file neve

OUTPUT: -

retVal:
- 0: a felcsatolás sikeres
- hibakód+$80: file nem .dsk file, vagy valami hiba történt

Unmount DSK ($93)

Lecsatolja a DSK Drive -ot és ha DSK Drive volt aktív, akkor beállítja az aktív drive-ot arra a drive -ra, ahonnan a .dsk drive fel volt csatolva. Innentől a REG_DRIVE_DSK_VALID regiszter 0-t fog visszaadni olvasásra.

INPUT: -

OUTPUT: -

retVal:
- 0: a lecsatolás sikeres
- hibakód+$80: nincs felcsatolt DSK Drive.

IO Műveletek emulátor készítőknek

Tudni érdemes, hogy van egy harmadik módja az alacsony szintű file műveleteknek, de azt mezei felhasználók nem tudják használni. Az egyes implementált TVC OS I/O műveleteknek olyan alacsony szintű kártya elérés kell, ami nem használja a TVC saját RAMját, azaz nincs hatással az éppen futó programra / TVC ROM függvényekre, még véletlenül se rontsa el a TVC belső állapotát.

Ezt úgy értem el, hogy van egy speciális RAM terület az IO ROM területen, amit csak akkor ér el a TVC-ben a Z80 CPU, ha a TVC IO ROM be van lapozva. Pl akkor, amikor épp egy TVC OS fgv hívás történik, mondjuk a LOAD során.

Ilyenkor a TVC és a kártya között a paraméterátadás ezen a speciális RAM területen történik. Ennek a RAM területnek a mérete 2kB, amiből 256 byte-ot a kártya saját célra használ, változók tárolására.

A fgv hívás úgy történik, hogy a ROM rutin a $D900 címtől kitölti az input paramétereket, lényegében pontosan ugyanúgy, ahol a függvényhívásoknál korábban részleteztük, majd a `REG_USB_MSC_CMD` regiszterbe beírja az megfelelő utasítás kódját. Mivel itt nincsenek speciális rutinok, a fileműveletek itt 1-től kezdődnek (szemben a fgv hívásokkal, ahol $81 -el kezdődnek), azaz a fgv kódnak $80 -al kevesebbet kell megadni, mint a fgv hívás során. Pl az `MSC_FOPENFILE` kódja a $01, az `MSC_FSEEK` pedig $0a.

Az input paramétert a *kártya* mindig a $1900 IO ROM címtől (*TVC* cím: $D900) várja, míg az outputot a $1a00 címtől (*TVC* cím: $DA00) adha vissza ugyanabban a formában, ahogyan azt a fgv hívások során is részleteztem.

Ebből már látható, h miért lassabb egy $1000 hosszú blokk beolvasása a TVC OS fgv -el, mint a `read_file_dest` hívással: azért, mert

1. a kártya által implementált TVC OS block read fgv csak 1kB os blokkmérettel dolgozik, azaz a $1000 méretű blokkot 4 blokk-olvasással tudja betölteni
2. minden olvasás után be kell másolnia megfelelő területre a TVC címterén belül. Ha ez a terület a TVC belső RAMja, akkor nincs választás, a TVC OS fgv-t kell használni. De amennyiben a kártya fastRAM terület a cél, akkor (is) 4szer kell másolnia a $DA00 területről a célterületre (ezt persze megcsinálja a kártya TVC OS alá implementált block read fgv-e), de ez elkerülhető, ha a `read_file_dest` fgv -t hívjuk, mert ekkor az adat azonnal a megadott címen fog megjelenni, a Z80 CPU-nak nem kell másolgatnia semmit sem.

Csak a teljesség kedvéért, a REG_USB_MSC_CMD regiszter által elfogadott kódok:

```
MSC_FOPENFILE = $01,
MSC_FCLOSEFILE,
```

```
    MSC_FREAD,
    MSC_FWRITE,
    MSC_FOPENDIR,
    MSC_FCLOSEDIR,
    MSC_FREADDIR,
    MSC_FWRITE_SRC,
    MSC_FREAD_DEST,
    MSC_FSEEK,
    MSC_FGETCWD,
    MSC_FCHDIR,
    MSC_FMKDIR,
    MSC_FDELETE,
    MSC_FRENAME,
    MSC_FSTAT,
    MSC_FSYNC,
    MSC_MOUNT_DSK,
    MSC_UMOUNT_DSK // = $13
```

És tisztázandó: a regiszter neve félrevezető, mert valójában nem csak az USB drive -on hajtja végre a kiadott parancsokat, hanem mindig az aktív drive -on (próbálja meg végrehajtani).

Köszönetnyilvánítás

Ez a kártya nem jött volna létre Kollár Zoltán mentorom korai TVC-s munkája - az ő sprite kártyája - nélkül. Ő adott inspirációt a kártyához, kezdő lökést az elektronikai tervezés (“bütykölés”) felé. Egyszer majd találkozunk.

Ez a kártya nem jött volna létre családom támogatása nélkül, akik elnézték (elnézik) nekem, hogy kivonom magam a családi életből egy-egy délutánra, éjszakára.

Ez a kártya sokkal lassabban (még ennél is...) jött volna létre Mikus Lóránt nélkül, aki nagyon sok tanáccsal és segítséggel látott el a fejlesztés során.

Ez a kártya nem jött volna létre a TVC közösség érdeklődése és biztatása nélkül.

Ez a dokumentum tele lenne hibával, elírással, pontatlanságokkal Kis Róbert nélkül, aki többször is lektoráltatta a saját AI szolgájával.

Mindenkinek nagyon köszönöm a segítséget és támogatást.

Vass Sándor, 2026